

**ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΗΛΕΚΤΡΟΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ
ΠΛΗΡΟΦΟΡΙΚΗΣ**



Τομέας: Υπολογιστική Υψηλών Επιδόσεων
Εργαστήριο: Εργαστήριο Πληροφοριακών Συστημάτων
Υψηλών Επιδόσεων

**Συγκριτική Ανάλυση και Πειραματική
Αξιολόγηση Συστημάτων Παράλληλου
Προγραμματισμού Βασισμένου σε
Εργασίες**

Διπλωματική Εργασία

του φοιτητή

Γεωργίου Τσάβου του Κωνσταντίνου

Αριθμός Μητρώου: 1084606

Επιβλέπων: Παναγιώτης Χατζηδούκας, Αναπληρωτής Καθηγητής,
Πανεπιστήμιο Πατρών

Πάτρα, Ιούνιος 2026

ΠΙΣΤΟΠΟΙΗΣΗ

Πιστοποιείται ότι η διπλωματική εργασία με θέμα:

**Συγκριτική Ανάλυση και Πειραματική Αξιολόγηση
Συστημάτων Παράλληλου Προγραμματισμού Βασισμένου
σε Εργασίες**

του φοιτητή του Τμήματος Μηχανικών Ηλεκτρονικών Υπολογιστών και
Πληροφορικής

Γεωργίου Τσάβου του Κωνσταντίνου

(Α.Μ.: 1084606)

παρουσιάστηκε δημόσια και εξετάστηκε επιτυχώς από την
τριμελή εξεταστική επιτροπή στις __/__/____.

Ο Επιβλέπων

Μέλος της Επιτροπής

Μέλος της Επιτροπής

Παναγιώτης

Χατζηδούκας

Αναπληρωτής Καθηγητής

Σπυρίδων

Σιούτας

Καθηγητής

Γεώργιος Παπαδημητρίου

Επίκουρος Καθηγητής

Στοιχεία Διπλωματικής Εργασίας

Θέμα:

Συγκριτική Ανάλυση και Πειραματική Αξιολόγηση
Συστημάτων Παράλληλου Προγραμματισμού Βασισμένου
σε Εργασίες

Φοιτητής:

Γεώργιος Τσάβος του Κωνσταντίνου

Ομάδα Επίβλεψης:

Παναγιώτης Χατζηδούκας

Αναπληρωτής Καθηγητής, Πανεπιστήμιο Πατρών

Εργαστήριο:

Εργαστήριο Πληροφοριακών Συστημάτων Υψηλών Επιδόσεων

Περίληψη

Λέξεις-κλειδιά: HPC, Διαργασίες, task-parallelization, heterogeneity

Abstract

Keywords: HPC, Διεργασίες, task-parallelization, heterogeneity

Ευχαριστίες

Περιεχόμενα

Συντομογραφίες	ix
Κατάλογος σχημάτων	x
Κατάλογος πινάκων	xi
1 Εισαγωγή	1
1.1 Σκοπός και Αντικείμενο της Εργασίας	1
1.2 Δομή της Εργασίας	2
2 Θεωρητικό Υπόβαθρο	5
2.1 Παραλληλοποίηση με Εργασίες	5
2.2 Χρονοδρομολόγηση Εργασιών	6
2.3 Αρχιτεκτονική NUMA	7
2.4 Προκλήσεις της Αρχιτεκτονικής NUMA στην παραλληλοποίηση με Εργασίες	9
2.5 Παραλληλοποίηση στην Python	10
3 Το Σύστημα StarPU	13
3.1 Τι είναι το StarPU	13
3.2 Η Αρχιτεκτονική του StarPU	14
3.2.1 Οντότητες Εργασίας	14
3.2.2 Οντότητες Εκτέλεσης	15
3.2.3 Οντότητες Δεδομένων	16
3.3 Υποσύστημα Εικονικής Κοινής Μνήμης (VSM)	16
3.3.1 Δομές και Διεπαφές Δεδομένων	16
3.3.2 Πρωτόκολλο Συνοχής MSI (Modified, Shared, Invalid)	17
3.3.3 Προηγμένες λειτουργίες πρόσβασης δεδομένων	18
3.3.4 Κατάτμηση Δεδομένων και Ιεραρχικά Φίλτρα	19
3.3.5 Ασύγχρονες Μεταφορές και Προανάγνωση (Prefetching)	20
3.4 Χρονοπρογραμματισμός Εργασιών	20

3.4.1	Μοντέλα Απόδοσης Πραγματικού Χρόνου	20
3.4.2	Πολτικές Χρονοπρογραμματισμού Εργασιών	21
3.5	Κύκλος Εκτέλεσης Εσωτερικών Εργατών	23
3.5.1	Ανάκτηση Εργασιών και Διαχείριση Καταστάσεων	23
3.5.2	Χαλαρός Συγχρονισμός και Αποφυγή Αδιεξόδων	23
3.6	Κατανεμημένη Εκτέλεση: MPI	24
3.6.1	Υποβολή Εργασιών σε Κατανεμημένο Περιβάλλον	24
3.6.2	Βελτιστοποιήσεις Επικοινωνίας MPI	25
3.7	Σημαντικά Υποσυστήματα του StarPU	25
3.7.1	Out-of-Core Εκτέλεση	25
3.7.2	Context Hypervisors και Παράλληλες Εργασίες	26
3.7.3	Ανοχή Σφαλμάτων (Fault Tolerance)	26
4	Η Διεπαφή StarPUPy	27
4.1	Τι είναι το StarPUPy	27
4.2	Ασύγχρονη Υποβολή Εργασιών και Διαχείριση Αντικειμένων Future . . .	27
4.3	Διαχείριση Μνήμης και Πρωτόκολλο Buffer	28
4.4	Παράλληλια και Αντιμετώπιση του GIL	29
4.5	Παράδειγμα Χρήσης του StarPUPy	30
4.5.1	Ολοκληρωμένος Κώδικας Παραδείγματος	32
5	Το Σύστημα torepy	35
5.1	Τι είναι το torepy	35
5.2	Αρχιτεκτονική του torepy	36
5.2.1	Δομή Εργασιών και Διαχείριση Αποτελεσμάτων	37
5.2.2	Ο Ρόλος και η Λειτουργία του Νήματος Εξυπηρετητή (Server Thread)	37
5.3	Μηχανισμός Εξισορρόπησης Φόρτου	38
5.4	Προγραμματιστική Διεπαφή (API) και Υποστήριξη Κληρονομημένου Κώ- δικα (SPMD)	39
5.4.1	Βασικές Ρουτίνες Διαχείρισης (Library Routines)	39
5.4.2	Μοντέλο Εκτέλεσης SPMD	41
5.5	Παράδειγμα Χρήσης του torepy	41
5.5.1	Ολοκληρωμένος Κώδικας Παραδείγματος	43
	Βιβλιογραφία	45

Συντομογραφίες

HPC	High Performance Computing
CPU	Central Processing Unit
PU	Processing Unit
GPU	Graphics Processing Unit
FPGA	Field-Programmable Gate Array
FCFS	First-Come-First-Served
RR	Round Robin
NUMA	Non-Uniform Memory Access
SMP	Symmetric Multi-Processing
UMA	Uniform Memory Access
HEFT	Heterogeneous Earliest Finish Time
GIL	Global Interpreter Lock
PEP	Python Enhancement Proposal
VSM	Virtual Shared Memory
MPI	Message Passing Interface
API	Application Programming Interface
DAG	Directed Acyclic Graph
MSI	Modified, Shared, Invalid
MESI	Modified, Exclusive, Shared, Invalid
LRU	Least Recently Used
DMA	Direct Memory Access
SPMD	Single Program Multiple Data

Κατάλογος σχημάτων

2.1	Βήματα παραλληλοποίησης ενός αλγορίθμου με εργασίες. (Πηγή: [1]) . .	6
2.2	Αρχιτεκτονική Ομοιόμορφης Πρόσβασης Μνήμης (UMA) ή Συμμετρικής Πολυεπεξεργασίας (SMP). (Πηγή: [5])	8
2.3	Αρχιτεκτονική Μη Ομοιόμορφης Πρόσβασης Μνήμης (NUMA). (Πηγή: [5])	8

Κατάλογος πινάκων

3.1	Καταστάσεις του πρωτοκόλλου MSI	18
3.2	Μοντέλα Απόδοσης στο StarPU	21
4.1	Επιλογές για την συνάρτηση task_submit του StarPUPy	28

Κεφάλαιο 1

Εισαγωγή

1.1 Σκοπός και Αντικείμενο της Εργασίας

Η ραγδαία εξέλιξη της υπολογιστικής επιστήμης και η ανάγκη για την ανάλυση όλο και μεγαλύτερου όγκου δεδομένων έχουν καταστήσει τον παράλληλο προγραμματισμό αναπόσπαστο κομμάτι της σύγχρονης έρευνας και τεχνολογίας. Καθώς η φυσική αύξηση της συχνότητας των επεξεργαστών (ρολόι) έχει φτάσει στα όριά της, η σύγχρονη υπολογιστική ισχύς πηγάζει πλέον κυρίως από αρχιτεκτονικές πολλαπλών πυρήνων (multi-core) και συστήματα συμμετρικής ή μη-συμμετρικής πολυεπεξεργασίας. Παράλληλα, γλώσσες υψηλού επιπέδου όπως η Python κυριαρχούν στους επιστημονικούς υπολογισμούς λόγω της ευκολίας χρήσης τους και του πλούσιου οικοσυστήματος βιβλιοθηκών τους, όμως η αποτελεσματική εκμετάλλευση του σύγχρονου υλικού (hardware) μέσω αυτής αποτελεί μια μείζονα πρόκληση, κυρίως λόγω εγγενών περιορισμών της γλώσσας στον ταυτόχρονο προγραμματισμό.

Αντικείμενο της παρούσας διπλωματικής εργασίας είναι η συγκριτική ανάλυση και πειραματική αξιολόγηση δύο διαφορετικών συστημάτων παράλληλου προγραμματισμού βασισμένου σε εργασίες (task-based parallel programming). Το μοντέλο αυτό προσφέρει ένα υψηλό επίπεδο αφάιρησης, όπου το πρόγραμμα διασπάται σε ανεξάρτητες μονάδες εκτέλεσης, τις "εργασίες" (tasks). Ο προγραμματιστής ορίζει τις εργασίες και τις μεταξύ τους εξαρτήσεις, ενώ το σύστημα χρόνου εκτέλεσης (runtime system) αναλαμβάνει δυναμικά να τις δρομολογήσει στους διαθέσιμους πυρήνες, να διαχειριστεί τη μνήμη και να εξισορροπήσει τον φόρτο εργασίας, απαλλάσσοντας τον χρήστη από τον χειροκίνητο και επίπονο έλεγχο των νημάτων.

Σκοπός της εργασίας είναι να μελετηθεί πώς τα συστήματα αυτά ανταποκρίνονται στις σύγχρονες προκλήσεις του υλικού και του λογισμικού. Η πρώτη μεγάλη πρόκληση αφορά τις αρχιτεκτονικές **NUMA (Non-Uniform Memory Access)**, οι οποίες κυριαρχούν στα συστήματα υψηλών επιδόσεων (HPC). Σε αυτές, η μνήμη είναι χωρισμένη σε τοπικά τμήματα ανά επεξεργαστή, με αποτέλεσμα η πρόσβαση σε απομακρυσμένα τμήματα μνήμης να προκαλεί καθυστερήσεις και συμφόρηση. Τα task-based συστήματα πρέπει, επομένως, να έχουν επίγνωση της τοπολογίας (NUMA-awareness) για να διατηρούν την

τοπικότητα των δεδομένων. Η δεύτερη και σημαντικότερη πρόκληση στο περιβάλλον της Python είναι το **Global Interpreter Lock (GIL)**, ένας μηχανισμός που επιτρέπει την εκτέλεση κώδικα Python από ένα μόνο νήμα ανά πάσα στιγμή. Σε *cpu-bound* εφαρμογές, η χρήση απλών νημάτων οδηγεί σε *GIL thrashing* (διαρκείς εναλλαγές για την απόκτηση του lock), προκαλώντας κατακερματισμό της χρήσης της CPU (ακόμα και στο 1.5%) και ακυρώνοντας τον παραλληλισμό.

Για την αντιμετώπιση αυτών των προβλημάτων, η εργασία εξετάζει και συγκρίνει δύο πλατφόρμες με θεμελιωδώς διαφορετικές αρχιτεκτονικές φιλοσοφίες:

- **StarPU (και η διεπαφή StarPUPY):** Ένα σύστημα χαμηλού επιπέδου σε C, σχεδιασμένο για *shared memory* και ετερογενή συστήματα. Θα επικεντρωθούμε στην Python διεπαφή του, StarPUPY για την πειραματική αξιολόγηση.
- **torcpy:** Ένα Python-native σύστημα βασισμένο στο κατανεμημένο πρότυπο MPI (μέσω της `mpi4py`), ιδανικό για κατενευμένα συστήματα.

Η πειραματική αξιολόγηση και η συνεισφορά της εργασίας εστιάζουν στην υλοποίηση και σύγκριση των δύο συστημάτων πάνω σε τρεις αντιπροσωπευτικές *cpu-bound* εφαρμογές: έναν αλγόριθμο βελτιστοποίησης (CMA-ES με επίλυση Poisson PDE), μια ροή επεξεργασίας εικόνας (Gabor Filters), και μια προσομοίωση Monte Carlo (Basket Options). Μέσα από αυτές τις εφαρμογές, η εργασία ποσοτικοποιεί τον χρόνο εκτέλεσης σε συνθήκες ισχυρού παραλληλισμού (64 πυρήνες), αναδεικνύοντας τα όρια και τις βέλτιστες πρακτικές χρήσης του κάθε συστήματος.

1.2 Δομή της Εργασίας

Η παρούσα διπλωματική εργασία είναι οργανωμένη σε επτά συνολικά κεφάλαια, ακολουθώντας μια λογική πορεία από τη θεωρητική θεμελίωση στην πρακτική εφαρμογή και την ανάλυση:

- **Κεφάλαιο 2 -- Θεωρητικό Υπόβαθρο:** Παρουσιάζεται το θεωρητικό υπόβαθρο γύρω από την παραλληλοποίηση με εργασίες, την ετερογένεια και τις προκλήσεις των αρχιτεκτονικών NUMA, την παραλληλοποίηση σε Python και το GIL.
- **Κεφάλαιο 3 -- Το Σύστημα StarPU:** Αναλύονται οι δυνατότητες και η αρχιτεκτονική του StarPU σε περιβάλλον C, καθώς και ο τρόπος διαχείρισης των πόρων.
- **Κεφάλαιο 4 -- Η Διεπαφή StarPUPY:** Εξετάζεται η μεταφορά του StarPU στην Python, οι μέθοδοι διαχείρισης του GIL και οι ιδιαιτερότητες του χρονοπρογραμματισμού του.

- **Κεφάλαιο 5 -- Το Σύστημα torcpy:** Αναλύεται το μοντέλο εκτέλεσης του torcpy, το API του, ο τρόπος δρομολόγησης των tasks και η χρήση του MPI για την απομόνωση διεργασιών.
- **Κεφάλαιο 6 -- Εφαρμογές και Μεθοδολογία:** Περιγράφεται αναλυτικά ο κώδικας και η μαθηματική δομή των τριών εφαρμογών (CMA-ES, Image Processing, Monte Carlo), καθώς και ο τρόπος λήψης των μετρήσεων.
- **Κεφάλαιο 7 -- Πειραματική Αξιολόγηση και Συμπεράσματα:** Παρουσιάζονται τα διαγράμματα και τα αποτελέσματα των μετρήσεων, αναλύονται τα φαινόμενα saturation και κλιμάκωσης, και συνοψίζονται τα τελικά συμπεράσματα της σύγκρισης.

Κεφάλαιο 2

Θεωρητικό Υπόβαθρο

2.1 Παραλληλοποίηση με Εργασίες

Στον χώρο του HPC, η παραλληλοποίηση με εργασίες (task parallelism) είναι μια προσέγγιση, η πλέον πιο διαδεδομένη, που επιτρέπει την εκτέλεση πολλαπλών εργασιών ταυτόχρονα, όπου κάθε εργασία μπορεί να είναι ανεξάρτητη ή να έχει εξαρτήσεις από άλλες εργασίες. Αυτή η μέθοδος είναι ιδιαίτερα χρήσιμη όταν οι εργασίες έχουν διαφορετικά μεγέθη ή απαιτούν διαφορετικούς πόρους. Αυτό έχει ως αποτέλεσμα την καλύτερη εκμετάλλευση των διαθέσιμων πόρων του συστήματος και τη μείωση του χρόνου εκτέλεσης.

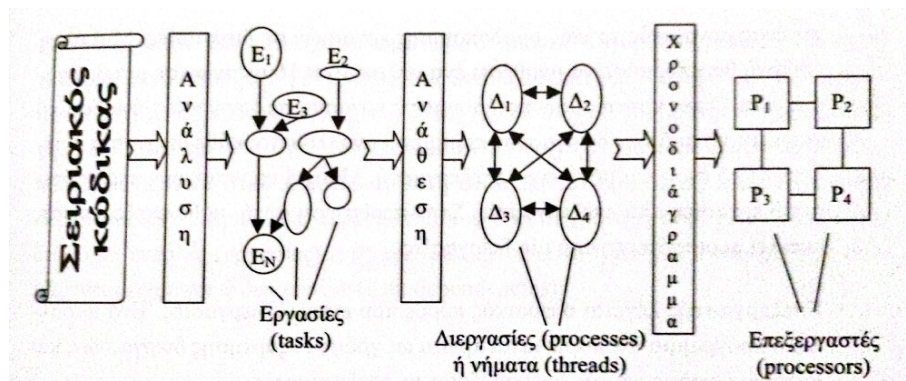
Η μετατροπή ενός αλγορίθμου για εκτέλεση σε ένα περιβάλλον παραλληλοποίησης με εργασίες απαιτεί διάφορα βήματα. Αρχικά, η υλοποίηση και η μελέτη του αλγορίθμου σε σειριακή μορφή σε μια γλώσσα προγραμματισμού θα βοηθήσει στην κατανόηση της λειτουργίας του και στην αναγνώριση των τμημάτων που μπορούν να παραλληλοποιηθούν. Ιδανικά αυτό θα θέλαμε να γίνεται αυτόματα από τον μεταγλωτιστή και το λειτουργικό σύστημα, αλλά στην πράξη απαιτείται η παρέμβαση του προγραμματιστή για να καθορίσει τις εργασίες και τις εξαρτήσεις τους [1].

Τα βήματα που πρέπει να λάβει υπόψη ο προγραμματιστής στην διαδικασία παραλληλοποίησης ενός αλγορίθμου, είναι τα εξής [1]:

1. **Ανάλυση του Αλγορίθμου σε Εργασίες (tasks):** Ο αλγόριθμος πρέπει να αναλυθεί για να εντοπιστούν τα τμήματα που μπορούν να εκτελεστούν ανεξάρτητα. Αυτό μπορεί να περιλαμβάνει την ανάλυση των βρόχων, των συναρτήσεων και των δεδομένων που χρησιμοποιούνται.
2. **Ανάθεση σε Διεργασίες (processes) ή Νήματα (threads):** Οι εργασίες πρέπει να ανατεθούν σε διαφορετικές διεργασίες (ή νήματα) ή να εκτελεστούν σε διαφορετικούς πόρους, με βάση την διαθεσιμότητα και τις απαιτήσεις του κάθε ενός.
3. **Χρονοδρομολόγηση των Διεργασιών (scheduling):** Η χρονοδρομολόγηση των εργασιών είναι ένα σύνθετο βήμα που αναλύεται στα εξής:

- (α') **Ανάθεση κάθε Διεργασίας σε ένα Πόρο:** Κάθε διεργασία πρέπει να ανατεθεί σε έναν πόρο, όπως ένας επεξεργαστής ή ένας πυρήνας, με βάση τις απαιτήσεις της εργασίας και τη διαθεσιμότητα των πόρων.
- (β') **Απόφαση για την Εκτέλεση των Εργασιών:** Δηλαδή, πότε θα εκτελεστεί κάθε εργασία (task) στον συγκεκριμένο πόρο που του αντιστοιχίστηκε στο βήμα 3α'.

Τα παραπάνω βήματα παραλληλοποίησης ενός αλγορίθμου με εργασίες φαίνονται και στο παρακάτω Σχήμα 2.1.



Σχήμα 2.1: Βήματα παραλληλοποίησης ενός αλγορίθμου με εργασίες. (Πηγή: [1])

2.2 Χρονοδρομολόγηση Εργασιών

Όπως είδαμε στην προηγούμενη ενότητα, η χρονοδρομολόγηση των εργασιών είναι το πιο σύνθετο βήμα από τα υπόλοιπα, το οποίο στην σημερινή εποχή γίνεται όλο και πιο δύσκολο λόγω της αυξανόμενης πολυπλοκότητας των συστημάτων HPC και των εφαρμογών που εκτελούνται σε αυτά.

Η χρονοδρομολόγηση των εργασιών περιλαμβάνει την ανάθεση κάθε διεργασίας σε έναν πόρο, όπως ένας επεξεργαστής ή ένας πυρήνας, με βάση τις απαιτήσεις της εργασίας και τη διαθεσιμότητα των πόρων. Επιπλέον, πρέπει να ληφθεί υπόψη η απόφαση για το πότε θα εκτελεστεί κάθε εργασία (task) στον συγκεκριμένο πόρο που του αντιστοιχίστηκε. Αυτό καθιστά την επιλογή μιας αποτελεσματικής στρατηγικής (ή πολιτικής) χρονοδρομολόγησης κρίσιμη για την απόδοση των εφαρμογών σε περιβάλλοντα HPC.

Παλαιότερα ήταν αρκετό να γίνεται η χρήση απλών πολιτικών χρονοδρομολόγησης, όπως η First-Come-First-Served (FCFS) ή η Round Robin (RR), για να επιτευχθεί ικανοποιητική απόδοση. Ωστόσο, με την αύξηση της πολυπλοκότητας των συστημάτων και των εφαρμογών, αυτές οι απλές πολιτικές δεν είναι πλέον επαρκείς. Σήμερα, απαιτούνται πιο εξελιγμένες στρατηγικές χρονοδρομολόγησης που λαμβάνουν υπόψη παράγοντες όπως οι

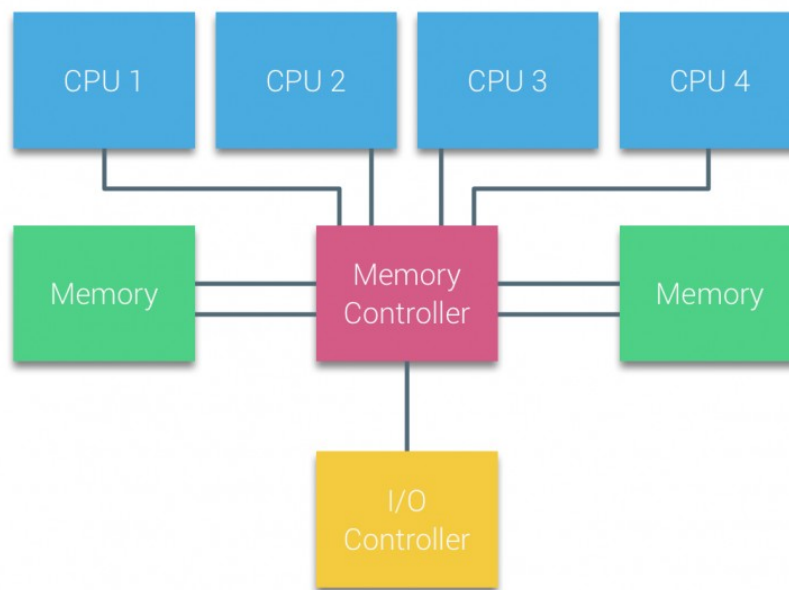
εξαρτήσεις μεταξύ των εργασιών, η εκτίμηση του χρόνου εκτέλεσης κάθε εργασίας, η δυναμική κατάσταση των πόρων και η τοπικότητα των δεδομένων κάθε εργασίας στην μνήμη, διότι πλέον τα συστήματα είναι πολύπολκα εισάγοντας έννοιες ετερογένειας (heterogeneity) και πολλαπλών επεξεργαστηκών μονάδων, όπως οι αρχιτεκτονικές **NUMA**.

Οπότε θέλουμε η χρονοδρομολόγηση των εργασιών να γίνεται δυναμικά είτε από το λειτουργικό σύστημα είτε από το περιβάλλον εκτέλεσης (runtime environment) με βάση τις τρέχουσες συνθήκες του συστήματος και των εργασιών. Αυτό μπορεί να επιτευχθεί μέσω της χρήσης τεχνικών όπως **Work Stealing**, **HEFT** κ.α. που λαμβάνουν υπόψη τους διάφορους παράγοντες που αναφέρθηκαν προηγουμένως. Τέτοιες τεχνικές υλοποιούνται όπως θα δουμε σε παρακάτω κεφάλαια και από τη **torc** και τη **StarPU** [2, 3] στο επίπεδο του περιβάλλοντος εκτέλεσης.

2.3 Αρχιτεκτονική NUMA

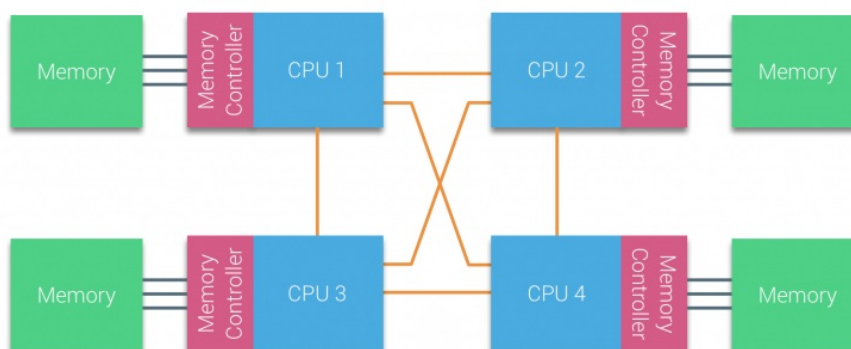
Όπως αναφέρθηκε και στην προηγούμενη ενότητα, η πολυπλοκότητα των συστημάτων **HPC** έχει αυξηθεί λόγω της διαρκούς εξέλιξης τους και ανάγκης για επίλυση πιο σύνθετων επιστημονικών και βιομηχανικών προβλημάτων. Ιστορικά, ωστόσο, το μεγαλύτερο εμπόδιο στην επίτευξη της μέγιστης δυνατής υπολογιστικής απόδοσης δεν ήταν η ταχύτητα των ίδιων των επεξεργαστών, αλλά ο περιορισμός που είναι ευρέως γνωστός ως το «εμπόδιο του Von Neumann» (Von Neumann bottleneck). Το φαινόμενο αυτό περιγράφει την εγγενή ανισορροπία μεταξύ της ταχύτητας επεξεργασίας των Κεντρικών Μονάδων Επεξεργασίας (**Central Processing Unit - CPU**) και της ταχύτητας μεταφοράς των δεδομένων από και προς την κύρια μνήμη του συστήματος, γεγονός που συχνά αφήνει τους επεξεργαστές σε κατάσταση αναμονής, στερημένους από τα απαραίτητα δεδομένα για την εκτέλεση των υπολογισμών [4].

Αρχικά, για να αντιμετωπιστή αυτό το πρόβλημα, οι σχεδιαστές συστημάτων υιοθέτησαν μια αρχιτεκτονική που ονομάζεται Ομοιόμορφη Πρόσβαση Μνήμης (**Uniform Memory Access - UMA**), η οποία είναι γνωστή και ως Συμμετρική Πολυεπεξεργασία (**Symmetric Multi-Processing - SMP**), όπου πολλοί επεξεργαστές μοιράζονται την ίδια φυσική μνήμη, όπως φαίνεται και στο Σχήμα 2.2. Το βασικό πλεονέκτημα αυτής της προσέγγισης ήταν η απλότητά της από πλευράς προγραμματισμού. Όλοι οι επεξεργαστές λάμβαναν απολύτως ισότιμη και ομοιόμορφη πρόσβαση (ίσο χρόνο απόκρισης) σε οποιαδήποτε θέση της μνήμης. Ωστόσο, καθώς ο αριθμός των επεξεργαστών σε ένα σύστημα αυξανόταν, η συνολική απόδοση του συστήματος άρχισε να υποφέρει λόγω της αυξημένης συμφόρησης στη μνήμη [5].



Σχήμα 2.2: Αρχιτεκτονική Ομοιόμορφης Πρόσβασης Μνήμης (UMA) ή Συμμετρικής Πολυεπεξεργασίας (SMP). (Πηγή: [5])

Για την επίλυση του παραπάνω προβλήματος αναπτύχθηκε μια νέα αρχιτεκτονική που ονομάζεται Μη Ομοιόμορφη Πρόσβαση Μνήμης (**Non-Uniform Memory Access - NUMA**), όπου κάθε επεξεργαστής έχει τη δική του τοπική μνήμη, αλλά μπορεί επίσης να έχει πρόσβαση στη μνήμη άλλων επεξεργαστών. Παράδειγμα αυτής της αρχιτεκτονικής φαίνεται στο Σχήμα 2.3. Σε αυτή την αρχιτεκτονική, η πρόσβαση στη τοπική μνήμη ενός επεξεργαστή είναι ταχύτερη από την πρόσβαση στη μνήμη άλλων επεξεργαστών, γεγονός που εισάγει μια μη ομοιόμορφη συμπεριφορά στην πρόσβαση στη μνήμη. Η αρχιτεκτονική **NUMA** επιτρέπει την καλύτερη κλιμάκωση των συστημάτων με πολλούς επεξεργαστές, αλλά απαιτεί επίσης πιο περίπλοκες στρατηγικές χρονοδρομολόγησης και διαχείρισης μνήμης για να μεγιστοποιηθεί η απόδοση των εφαρμογών που εκτελούνται σε αυτά τα συστήματα [5, 6].



Σχήμα 2.3: Αρχιτεκτονική Μη Ομοιόμορφης Πρόσβασης Μνήμης (NUMA). (Πηγή: [5])

2.4 Προκλήσεις της Αρχιτεκτονικής NUMA στην παραλληλοποίηση με Εργασίες

Η φαινομενική απλότητα που προσφέρει η λογική αφαίρεση του μοντέλου των εργασιών έρχεται σε άμεση σύγκρουση με την εξαιρετικά πολύπλοκη φυσική πραγματικότητα του υλικού **NUMA**. Η εγγενής απόκρυψη των αρχιτεκτονικών λεπτομερειών από τον προγραμματιστή σημαίνει ότι το σύστημα χρόνου εκτέλεσης (runtime system) καλείται να λάβει κρίσιμες αποφάσεις σχετικά με την τοποθέτηση δεδομένων και τη δρομολόγηση των εργασιών, χωρίς πάντα να διαθέτει την πλήρη εικόνα της υποκείμενης τοπολογίας. Όταν τα αυτόματα συστήματα δρομολόγησης κατανέμουν εργασίες χωρίς επίγνωση της φυσικής τοποθεσίας των δεδομένων (**NUMA-oblivious scheduling**), οι επιπτώσεις στην απόδοση είναι καταστροφικές. Αυτή η αναντιστοιχία οδηγεί σε εκθετική αύξηση της κίνησης στους διαύλους διασύνδεσης (interconnect traffic), ακραία υποβάθμιση του εύρους ζώνης και αδυναμία γραμμικής κλιμάκωσης, διαμορφώνοντας ένα τοπίο στο οποίο η συγκριτική ανάλυση και η πειραματική αξιολόγηση των σύγχρονων συστημάτων παράλληλου προγραμματισμού καθίσταται θεμελιώδους σημασίας για την ακαδημαϊκή έρευνα [7].

Προβλήματα όπως τα παραπάνω καλούνται να λύσουν διάφορες τεχνικές που έχουν αναπτυχθεί για την βελτιστοποίηση της απόδοσης σε συστήματα **NUMA**. Μία τέτοια τεχνική είναι η **NUMA-aware Work Stealing (NUMA-WS)**, η οποία τροποποίησε τον πυρήνα του Intel Cilk Plus εφαρμόζοντας ολιστικά την αρχή του **Work-First**¹. Αντί να βασίζεται σε τεχνικές τυχαίας κλοπής που οδηγούν σε μεταναστεύσεις μεγάλου κόστους, το **NUMA-WS** προτείνει τη μηχανική ώθησης (work pushing). Το σύστημα αναθέτει στον εργάτη την προσπάθεια τοποθέτησης ενός υπολογιστικού πλαισίου (full frame) ακριβώς δίπλα στα δεδομένα του, εκβιάζοντας τη δημιουργία τοπικότητας. Το **NUMA-WS** για να αποφύγει να επιβαρύνει πολύ το τρέχων νήμα-εργάτη εφαρμόζει ώθηση δεδομένων μόνο σε τρία συγκεκριμένα σενάρια: κατά την εκτέλεση πράξεων συγχρονισμού, κατά την επιστροφή θυγατρικού νήματος σε αδρανές γονικό, και μετά από επιτυχή κλοπή εργασίας. Με αυτόν τον τρόπο, σύμφωνα με τη φιλοσοφία **Work-First**, το κόστος της βελτίωσης της τοπικότητας **NUMA** δεν επιβαρύνει ποτέ το ενεργό νήμα-εργάτη, αλλά αποσβένεται αποκλειστικά στον αδρανή νήμα-κλέφτη. Το «κατώφλι ώθησης» εγγυάται ότι οι συνολικές λειτουργίες παραμένουν σταθερές και ελεγχόμενες, αποτρέποντας τεράστιο φόρτο εργασίας. Στην πράξη, τα αποτελέσματα είναι εντυπωσιακά: ο πληθωρισμός έργου **Work Inflation**² μπορεί να μειωθεί σημαντικά στο **NUMA-WS**, με σημαντική βελτίωση της κλιμάκωσης σε συστήματα με 32 και πλέον πυρήνες [7].

Παρόμοιες τεχνικές έχουν αναπτυχθεί και για άλλες πλατφόρμες, όπως η **StarPU** [3]

¹Η αρχή του **Work-First** υποστηρίζει ότι νέες εργασίες πρέπει να εκτελούνται το συντομότερο δυνατόν.

²Το **Work Inflation** αναφέρεται στο φαινόμενο όπου ο συνολικός χρόνος εκτέλεσης που παράγουν όλα τα νήματα μαζί είναι μεγαλύτερος από την εργασία που θα εκτελούσε ένας μόνο επεξεργαστής σειριακά.

και η **torcpy** [2] που θα μελετηθούν στην παρούσα εργασία, οι οποίες υλοποιούν διάφορες στρατηγικές χρονοδρομολόγησης που λαμβάνουν υπόψη την τοπολογία **NUMA** για να βελτιώσουν την απόδοση των εφαρμογών που εκτελούνται σε τέτοια συστήματα.

2.5 Παραλληλοποίηση στην Python

Η **Python** είναι μια από τις πιο δημοφιλείς υψηλού επιπέδου γλώσσες προγραμματισμού σήμερα, γνωστή για την απλότητα, την ευκολία χρήσης της και το πλούσιο οικοσύστημα βιβλιοθηκών της.

Στην καρδιά της **Python (CPython)** βρίσκεται το **Gloabl Interpreter Lock (GIL)**, το οποίο είναι ένας μηχανισμός που επιτρέπει μόνο σε ένα νήμα λειτουργικού συστήματος να εκτελεί **Python bytecode** τη φορά, ακόμα και σε συστήματα με πολλούς επεξεργαστές. Αυτό σημαίνει ότι η παραλληλοποίηση με νήματα (threading) στην **Python** δεν μπορεί να εκμεταλλευτεί πλήρως τις δυνατότητες των πολυπύρηνων συστημάτων, περιορίζοντας την απόδοση των εφαρμογών που απαιτούν έντονη επεξεργασία [8, 9, 10]. Πιο συγκεκριμένα:

- **CPU-bound Εφαρμογές:** Για υπολογιστικά εντατικές εργασίες, το GIL λειτουργεί ως απόλυτο σημείο συμφόρησης, ακυρώνοντας εντελώς τα πλεονεκτήματα των πολυπύρηνων επεξεργαστών **NUMA**.
- **I/O-bound Εφαρμογές:** Για εργασίες που εξαρτώνται από αιτήματα δικτύου, λειτουργίες αρχείων κλπ, το GIL είναι εξαιρετικά αποδοτικό (τα νήματα απελευθερώνουν το GIL όταν περιμένουν).

Ωστόσο, υπάρχουν διάφορες βιβλιοθήκες και εργαλεία που έχουν αναπτυχθεί για να παρακάμψουν τους περιορισμούς του GIL και να επιτρέψουν την παραλληλοποίηση στην **Python** [11]. Μερικά από αυτά περιλαμβάνουν:

- **Multiprocessing Module:** Η βιβλιοθήκη **multiprocessing** επιτρέπει τη δημιουργία νέων διεργασιών, οι οποίες μπορούν να εκτελούνται παράλληλα και να εκμεταλλεύονται πολλούς επεξεργαστές, παρακάμπτοντας έτσι το GIL. Ωστόσο, η χρήση διεργασιών μπορεί να είναι πιο βαριά σε πόρους και να απαιτεί περισσότερη μνήμη σε σύγκριση με τα νήματα.
- **Threading Module:** Η βιβλιοθήκη **threading** επιτρέπει τη δημιουργία νήματων, αλλά λόγω του GIL, δεν είναι αποτελεσματική για CPU-bound εργασίες.
- **Asyncio Module:** Η βιβλιοθήκη **asyncio** παρέχει υποστήριξη για ασύγχρονο προγραμματισμό, επιτρέποντας την εκτέλεση πολλαπλών εργασιών (tasks) ταυτόχρονα χωρίς να μπλοκάρει το κύριο νήμα, αλλά δεν είναι κατάλληλη για CPU-bound εργασίες.

- **Concurrent.futures Module:** Η βιβλιοθήκη **concurrent.futures** παρέχει μια υψηλού επιπέδου διεπαφή για την εκτέλεση εργασιών (tasks) σε νήματα ή διεργασίες, επιτρέποντας την εύκολη παραλληλοποίηση, αλλά εξακολουθεί να υπόκειται στους περιορισμούς του GIL.

Κεφάλαιο 3

Το Σύστημα StarPU

3.1 Τι είναι το StarPU

Όπως έχουμε ήδη αναφέρει πολλές φορές είναι ότι τα συστήματα που χρησιμοποιούνται για εφαρμογές **HPC**, είναι πολύ περίπλοκα. Γι' αυτό το λόγο άλλωστε έχουμε αρχιτεκτονικές **NUMA**, που εισάγουν την έννοια της ετερογένειας (**heterogeneity**) στην πρόσβαση στη μνήμη αλλά και στην φύση της επεξεργαστικής μονάδας (**Processing Unit - PU**). Δηλαδή ένα σύστημα NUMA μπορεί να έχει επεξεργαστικές μονάδες που είναι πέρα από **CPUs**, αλλά και **GPUs** ή **FPGAs** κ.α. Η ετερογένεια αυτή κάνει την ανάπτυξη εφαρμογών για τέτοια συστήματα πολύ δύσκολη, καθώς ο προγραμματιστής πρέπει να λάβει υπόψη του τις ιδιαιτερότητες και την υποκείμενη αρχιτεκτονική κάθε τύπου επεξεργαστικής μονάδας και να διαχειριστεί την κατανομή των εργασιών και των δεδομένων με τρόπο που να μεγιστοποιεί την απόδοση.

Την λύση σε αυτό το πρόβλημα έρχεται να δώσει το **StarPU**, ένα runtime system που παρέχει ένα ενοποιημένο περιβάλλον για την ανάπτυξη εφαρμογών σε ετερογενή συστήματα. Το StarPU επιτρέπει στους προγραμματιστές να γράφουν κώδικα που μπορεί να εκτελεστεί σε διάφορους τύπους επεξεργαστικών μονάδων χωρίς να χρειάζεται να ανησυχούν για τις λεπτομέρειες της αρχιτεκτονικής κάθε μονάδας. Παρέχει μηχανισμούς για την κατανομή των εργασιών, τη διαχείριση της μνήμης και την επικοινωνία μεταξύ των επεξεργαστικών μονάδων, επιτρέποντας έτσι την αποτελεσματική εκμετάλλευση των πόρων του συστήματος [3, 12].

Η θεμελιώδης αρχή λειτουργίας του StarPU είναι να «κάνει δυναμικά ό,τι δεν μπορεί πλέον να γίνει στατικά», καθιστώντας τις εφαρμογές φορητές και ιδιαίτερα κλιμακώσιμες χωρίς να θυσιάζεται η απόδοση χαμηλού επιπέδου [13]. Προσφέροντας ένα πλούσιο API σε C, C++ και Fortran, καθώς και υποστήριξη pragma OpenMP, το σύστημα εκτέλεσης απαλλάσσει τον προγραμματιστή από την αντιμετώπιση των λεπτομερειών υλικού χαμηλού επιπέδου, επιτρέποντάς του να επικεντρωθεί αποκλειστικά στην αλγοριθμική λογική [12].

Η παρούσα ανάλυση παρέχει μια ολοκληρωμένη και εξαντλητική παρουσίαση του τι επιτυγχάνει το StarPU και των περίπλοκων εσωτερικών μηχανισμών που χρησιμοποιεί για να το πετύχει. Η αξιολόγηση αναλύει τις βασικές αρχιτεκτονικές οντότητες, το υποσύστημα

Εικονικής Κοινής Μνήμης (**Virtual Shared Memory - VSM**), αλγορίθμους χρονοπρογραμματισμού εργασιών βασισμένους σε μοντέλα απόδοσης πραγματικού χρόνου, τον κύκλο εκτέλεσης των εσωτερικών εργασιών του συστήματος εκτέλεσης και την επέκταση αυτών των παραδειγμάτων σε περιβάλλοντα συστάδων κατανεμημένης μνήμης μέσω **MPI** [14, 15].

3.2 Η Αρχιτεκτονική του StarPU

Η αρχιτεκτονική του StarPU αποτελείται από διάφορα βασικά στοιχεία που συνεργάζονται για να παρέχουν το περιβάλλον εκτέλεσης για ετερογενείς εφαρμογές. Οι τρεις κύριες κατηγορίες οντοτήτων είναι οι εξής: Οντότητες Εργασίας, Οντότητες Εκτέλεσης και Οντότητες Δεδομένων. Αυτός ο τριμερής διαχωρισμός επιτρέπει στο σύστημα χρόνου εκτέλεσης να αποσυνδέσει τον αλγοριθμικό ορισμό ενός προβλήματος από τη φυσική εκτέλεσή του στο υλικό [16].

3.2.1 Οντότητες Εργασίας

Στην καρδιά του προγραμματιστικού μοντέλου του StarPU βρίσκονται τα **Codelets** (struct starpu_codelet). Ένα codelet ενσωματώνει έναν υπολογιστικό αλγόριθμο (ή πυρήνα υπολογισμού) και παρέχει πολλαπλές υλοποιήσεις για τις διάφορες διαθέσιμες επεξεργαστικές μονάδες (**PU**s) με βάση την υποκείμενη αρχιτεκτονική. Για παράδειγμα, ένα codelet σχεδιασμένο για την κλιμάκωση ενός διανύσματος μπορεί να περιέχει έναν απλό δείκτη συνάρτησης στην C για την εκτέλεση σε CPU, μια υλοποίηση CUDA για GPU και μια υλοποίηση OpenCL για συσκευές AMD. Η δομή του codelet επισημαίνει ποιο υλικό (hardware) υποστηρίζει μέσω μιας μάσκας bit (π.χ. STARPU_CPU | STARPU_CUDA) και υποδεικνύει τον αριθμό των buffer δεδομένων που χειρίζεται (nbuffers). Ενοποιώντας τις διάφορες υλοποιήσεις σε ένα ενιαίο codelet, το StarPU επιτρέπει στο σύστημα εκτέλεσης να επιλέξει δυναμικά την καταλληλότερη υλοποίηση με βάση τη διαθεσιμότητα των πόρων και την τοπικότητα των δεδομένων [12, 16, 17].

Η δεύτερη οντότητα εργασίας είναι τα **Tasks**. Ένα task είναι μια συγκεκριμένη εμφάνιση ενός codelet που έχει δεσμευτεί με ένα συγκεκριμένο σύνολο δεδομένων. Ένα task λειτουργεί ως μεσολαβητής μεταξύ του συστήματος χρόνου εκτέλεσης (runtime system) και της εφαρμογής. Κατά την υποβολή ενός task μέσω των βοηθητικών συναρτήσεων starpu_task_submit ή starpu_task_insert, ο προγραμματιστής καθορίζει το codelet, τα δεδομένα πάνω στα οποία θα εκτελεστεί (μέσω **Data Handles**) και τους τρόπους πρόσβασης (π.χ. STARPU_R, STARPU_W, STARPU_RW). Επιπλέον, η δομή του task υποστηρίζει προαιρετικά πεδία που παρέχουν υποδείξεις προς τον χρονοπρογραμματιστή, όπως επίπεδα προτεραιότητας, χρώμα εκτέλεσης για προφίλ και συναρτήσεις επανάκλησης

(callback_func) που ενεργοποιούνται μετά την επιτυχή ολοκλήρωση του task. Τα Tasks είναι εκ φύσεως ασύγχρονα από προεπιλογή, ωστόσο οι προγραμματιστές μπορούν να επιβάλουν σύγχρονη εκτέλεση ορίζοντας task->synchronous = 1 [12, 16, 17].

Η τελευταία οντότητα εργασίας είναι τα **Jobs**. Ένα job είναι μια δομή που χρησιμοποιεί εσωτερικά το StarPU για να αναπαραστήσει ένα task. Είναι μια χαμηλού επιπέδου οντότητα που διασχίζει τον εσωτερικό γράφο χρονοπρογραμματισμού. Ο διαχωρισμός μεταξύ tasks και jobs δίνει τη δυνατότητα για εξελιγμένες λειτουργίες όπως η παράλληλη εκτέλεση εργασιών, όπου μια μόνη δομή job μπορεί να συντονίσει πολλαπλές δομές παράλληλων tasks αντιστοιχισμένες σε μια ομάδα εργατών που χρησιμοποιούν παραλληλοποίηση σε επίπεδο νημάτων (όπως OpenMP). Η δομή job περιέχει εσωτερικά μεταδεδομένα που απαιτούνται για την ανάλυση εξαρτήσεων, την ουρά προτεραιοτήτων και την παρακολούθηση μοντέλων απόδοσης [12, 16, 17].

3.2.2 Οντότητες Εκτέλεσης

Η κεντρική οντότητα εκτέλεσης στο StarPU είναι ο **Worker**. Ένας worker είναι πάντα ένα νήμα (thread) CPU που έχει ως μοναδική ευθύνη την διαχείριση μιας συγκεκριμένης μονάδας υπολογισμού (PU). Για παράδειγμα, σε ένα σύστημα με 4 CPU και 2 GPU, το StarPU θα δημιουργήσει 6 workers: 4 για τους CPUs και 2 για τις GPUs. Ένας worker αν σχετίζεται με μια CPU είναι υπεύθυνος για την εκτέλεση των tasks. Αλλιώς στην περίπτωση που σχετίζεται με μια GPU ή άλλου είδους PU, τότε δρα σαν διαχειριστής για την συγκεκριμένη PU και δεν εκτελεί τον υπολογισμό. Δηλαδή, συνέχεια ρωτά για νέα tasks, μεταφέρει δεδομένα μέσω της διαύλου (bus) και στέλνει εντολές προς εκτέλεση στην PU που διαχειρίζετε. Αυτή η σχεδιαστική επιλογή είναι πολύ σημαντική, διότι επιτρέπει στο StarPU να διασφαλίζει ότι η διάυλος και οι ουρές εντολών της συσκευής είναι μόνιμα κορεσμένα, αποκρύπτοντας αποτελεσματικά την καθυστέρηση υλικού [16, 12, 15].

Η επόμενη οντότητα εκτέλεσης είναι οι **Drivers**. Οι drivers είναι οι διαμεσολαβητές μεταξύ των workers και του υλικού. Δίνουν στους worker ένα API ώστε να μπορούν να αρχικοποιούν το εκάστοτε υλικό, να χαρτογραφούν την μνήμη, να στέλνουν τα tasks στο εκάστοτε PU και να καθαρίζουν τα αχρίαστα δεδομένα μετά την εκτέλεση. Οι StarPU υποστηρίζει drivers για διάφορα είδη υλικού: CPU, CUDA, OpenCL, HIP, ακόμη και MPI Master-Slave διατάξης [12, 16].

Η τελευταία οντότητα εκτέλεσης είναι τα **Scheduling Contexts**. Για να μπορεί να γίνει διαχείριση των σύνθετων τοπολογιών υλικού, οι workers οργανώνονται σε scheduling contexts. Ένα scheduling context δεσμεύει μία ομάδα από workers σε ένα συγκεκριμένο αλγόριθμο χρονοπρογραμματισμού (scheduling algorithm). Για παράδειγμα, σε ένα σύστημα με 4 CPU και 2 GPU, το StarPU επιτρέπει στον προγραμματιστή να δημιουργήσει δύο scheduling contexts: ένα για τους 4 workers των CPU που χρησιμοποιούν έναν αλγόριθμο

χρονοπρογραμματισμού βασισμένο σε προτεραιότητα (priority-based scheduling) και ένα άλλο για τους 2 workers των GPU που χρησιμοποιούν έναν αλγόριθμο χρονοπρογραμματισμού βασισμένο σε απόδοση (performance-based scheduling). Αυτή η οργάνωση επιτρέπει στο StarPU να προσαρμόζει την στρατηγική χρονοπρογραμματισμού ανάλογα με τα χαρακτηριστικά του υλικού και τις απαιτήσεις της εφαρμογής [12, 16].

3.2.3 Οντότητες Δεδομένων

Η βασική οντότητα δεδομένων στο StarPU είναι το **Data Handle**. Ένα data handle είναι μια αδιαφανής αναφορά υψηλού επιπέδου σε ένα κομμάτι των δεδομένων της εφαρμογής. Λειτουργεί ως "λογική ταυτότητα" για τα δεδομένα, επιτρέποντας έτσι στο StarPU να παρακολουθεί να τα αιτήματα πρόσβασης (ανάγνωση, εγγραφή, ανάγνωση-εγγραφή) που κάνουν τα tasks. Έτσι το StarPU είναι σε θέση να αναλύει τις εξαρτήσεις μεταξύ των tasks και να φτιάχνει αυτόματα το γράφημα εξερτίσεων (**DAG**) [12, 16].

Η δεύτερη οντότητα δεδομένων είναι το **Data Replicate**. Ένα data replicate είναι μια οντότητα χαμηλού επιπέδου που αντιπροσωπεύει ένα συγκεκριμένο φυσικό αντίγραφο των δεδομένων ενός data handle, το οποίο βρίσκεται αποθηκευμένο σε έναν συγκεκριμένο κόμβο μνήμης που τον διαχειρίζεται το StarPU. Για σε έναν σύγχρονο σύστημα HPC, κόμβοι μνήμης είναι η κύρια μνήμη RAM της CPU, η ενσωματωμένη μνήμη VRAM της GPU 0, η VRAM της GPU 1, ή ακόμα και ο σκληρός δίσκος. Ένα Data Replicate είναι η πραγματική, φυσική αποτύπωση των δεδομένων της εφαρμογής σε έναν από αυτούς τους χώρους. Τέλος, ένα data replicate δεν είναι εμφανές στην εφαρμογή αλλά είναι χρήσιμη μόνο για το StarPU όπως θα δούμε στην συνέχεια [12, 16].

3.3 Υποσύστημα Εικονικής Κοινής Μνήμης (VSM)

Σε ετερογενή συστήματα HPC, ο κύριος παράγοντας που περιορίζει την απόδοση είναι οι μεταφορές δεδομένων. Για να αξιοποιηθεί πλήρως το υλικό, ένα σύστημα χρόνου εκτέλεσης πρέπει να ελαχιστοποιήσει αυτές τις μεταφορές και να τις επικαλύψει με τους υπολογισμούς. Το StarPU το επιτυγχάνει εφαρμόζοντας την αφαίρεση της Εικονικής Κοινής Μνήμης (**Virtual Shared Memory - VSM**).

3.3.1 Δομές και Διεπαφές Δεδομένων

Όπως αναφέραμε και στην υποενότητα 3.2.3, το StarPU χρησιμοποιεί δύο βασικές οντότητες δεδομένων: τα **Data Handles** και τα **Data Replicates**. Ένα data handle επιστρέφεται όταν ο προγραμματιστής εγγράφει ένα κομμάτι δεδομένων στο σύστημα χρόνου

εκτέλεσης, έναντι του κλασικού τρόπου με pointers C. Η φυσική διάταξη και το αποτύπωμα μνήμης των δεδομένων περιγράφονται από μια διεπαφή δεδομένων (**Data Interface**). Το StarPU παρέχει αρκετές προκαθορισμένες διεπαφές δεδομένων για διάφορες δομές δεδομένων που συναντιούνται συχνά σε επιστημονικούς υπολογισμούς, όπως [12, 16]:

- **Variable/Scalar Interface:** Για μεμονωμένες αριθμητικές τιμές ή μικρές δομές σταθερού μεγέθους (starpu_variable_data_register).
- **Vector Interface:** Για μονοδιάστατους πίνακες με συνεχόμενα στοιχεία (starpu_vector_data_register).
- **Matrix & Block Interfaces:** Για δισδιάστατες και τρισδιάστατες πυκνές δομές δεδομένων, λαμβάνοντας υπόψη τις κύριες διαστάσεις (LD) για εξαγωγή υπο-πινάκων χωρίς περιορισμούς συνεχόμενης μνήμης (starpu_matrix_data_register).
- **Sparse Interfaces:** Ενσωματωμένη υποστήριξη για μορφές Compressed Sparse Row (CSR), Block Compressed Sparse Row (BCSR) και Coordinate (COO), ενεργοποιώντας σύνθετους επιλύτες αραιής γραμμικής άλγεβρας.
- **N-Dimensional Tensor Interface:** Για εργασίες βαθιάς μάθησης και πολυδιάστατους πίνακες.

Με την αφαίρεση των δεδομένων σε data handles, το StarPU μπορεί να παρεμβαίνει σε κάθε αίτημα πρόσβασης σε αυτά. Ένα data handle δεν αντιπροσωπεύει ένα μεμονωμένο φυσικό buffer, αλλά λειτουργεί ως ένα γενικό μηχανήμα κατάστασης που παρακολουθεί πολλαπλά data replicates. Πάνω σε αυτό βασίζεται ο μηχανισμός συνοχής MSI (Modified, Shared, Invalid) που θα αναλύσουμε στην επόμενη υποενότητα [12, 16].

3.3.2 Πρωτόκολλο Συνοχής MSI (Modified, Shared, Invalid)

Για να μπορεί το StarPU να διατηρεί την συνοχή των δεδομένων μεταξύ όλων των κόμβων μνήμης, χρησιμοποιεί ένα πρωτόκολλο συνοχής κρυφής μνήμης **MSI (Modified, Shared, Invalid)**. Αυτό το πρωτόκολλο κανονικά εφαρμόζεται συνήθως απευθείας σε επίπεδο υλικού (hardware-level), αλλά το StarPU το ανάγει σε επίπεδο λογισμικού (software-level) με την χρήση των προαναφερθέντων data handles και data replicates. Η λύση αυτή σε επίπεδο λογισμικού έναντι άλλων παρόμοιων λύσεων σε επίπεδο υλικού για συνοχή κρυφής μνήμης (π.χ. MESI¹ για x86 multi-core CPUs), είναι πιο αποτελεσματική και επιτυγχάνει πιο προβλέψιμη συνοχή δεδομένων, διότι πλέον το πρωτόκολλο εφαρμόζεται

¹Το πρωτόκολλο MESI (Modified, Exclusive, Shared, Invalid) είναι ένα πρωτόκολλο συνοχής κρυφής μνήμης παρόμοιο με το MSI.

στο μάκρο-επίπεδο δομών δεδομένων έναντι του μικρο-επιπέδου των cache-lines ¹ [18, 19].

Το πρωτόκολλο MSI του StarPU λειτουργεί ως εξής:

Κατάσταση	Μηχανισμός και Συνέπειες
Modified (M)	Το αντίγραφο δεδομένων σε αυτόν τον κόμβο μνήμης είναι το μοναδικό έγκυρο και ενημερωμένο. Όλα τα άλλα αντίγραφα σε άλλους κόμβους επισημαίνονται ως Invalid. Ενεργοποιείται όταν μια εργασία εκτελεστεί με STARPU_W ή STARPU_RW.
Shared (S)	Το αντίγραφο δεδομένων είναι έγκυρο και πανομοιότυπο με άλλα Shared αντίγραφα σε διαφορετικούς κόμβους. Πολλαπλές εργασίες σε διαφορετικές GPU μπορούν να εκτελούνται ταυτόχρονα, διαβάζοντας από τα τοπικά τους αντίγραφα.
Invalid (I)	Το αντίγραφο δεδομένων είναι παρωχημένο ή μη αρχικοποιημένο. Εάν μια εργασία που ανατέθηκε σε αυτόν τον κόμβο απαιτεί τα δεδομένα, το σύστημα εκτέλεσης πρέπει να εκκινήσει μεταφορά (π.χ. DMA) από τον κόμβο που κατέχει το Modified ή Shared αντίγραφο.

Πίνακας 3.1: Καταστάσεις του πρωτοκόλλου MSI

Σημαντικό είναι ότι τα δεδομένα δεν διαγράφονται επιθετικά μόλις ολοκληρωθεί μια εργασία. Τα αντίγραφα παραμένουν στην κρυφή μνήμη του κόμβου μνήμης σε κατάσταση Κοινή (Shared) ή Τροποποιημένη (Modified). Εάν μια επόμενη εργασία που έχει προγραμματιστεί στο ίδιο PU ζητήσει το ίδιο data handle, τα δεδομένα βρίσκονται ήδη εκεί, με αποτέλεσμα μηδενικό κόστος μεταφοράς μνήμης. Η διαγραφή δεδομένων συμβαίνει μόνο όταν εξαντληθεί η φυσική μνήμη της συσκευής, οπότε το StarPU χρησιμοποιεί ευριστική των Λιγότερο Πρόσφατα Χρησιμοποιημένων (**LRU — Least Recently Used**) ² για να εκκαθαρίσει παρωχημένα δεδομένα πίσω στην κύρια μνήμη RAM ή στο δίσκο. Οι εφαρμογές μπορούν επίσης να παρέχουν υποδείξεις στο σύστημα χρόνου εκτέλεσης χρησιμοποιώντας τη συνάρτηση `starpu_data_wont_use()`, επιταχύνοντας την εκκαθάριση των συγκεκριμένων data handles [17, 18, 20].

3.3.3 Προηγμένες λειτουργίες πρόσβασης δεδομένων

Πέρα από τις τυπικές λειτουργίες Ανάγνωσης (Read) και Εγγραφής (Write), το StarPU εισάγει εξαιρετικά βελτιστοποιημένα παραδείγματα πρόσβασης προσαρμοσμένα για μαθηματικούς αλγόριθμους υψηλών επιδόσεων [12, 16]:

¹ Τα cache lines είναι οι μικρότερες μονάδες δεδομένων που μπορούν να μεταφερθούν μεταξύ της κύριας μνήμης και της κρυφής μνήμης.

² Η ευριστική LRU (Least Recently Used) είναι μια μέθοδος διαχείρισης μνήμης που εκκαθαρίζει τα δεδομένα που δεν έχουν χρησιμοποιηθεί για το μεγαλύτερο χρονικό διάστημα.

- **STARPU_REDUX:** Λειτουργώντας παρόμοια με τις reduction μεθόδους του OpenMP, αυτή η λειτουργία επιτρέπει σε πολλαπλά tasks να εκτελούνται ταυτόχρονα στο ίδιο data handle σε διαφορετικούς κόμβους. Το StarPU εκχωρεί δυναμικά buffers αρχικοποίησης ανά worker, επιτρέπει σε παράλληλα tasks να γράφουν ταυτόχρονα χωρίς κλείδωμα, και στη συνέχεια προγραμματίζει αυτόματα ένα δέντρο αναγωγής (reduction tree) από επακόλουθα tasks για τη συγχώνευση των διαφορετικών buffers στο τελικό αποτέλεσμα. Αυτό εξαλείφει πλήρως την ψευδή σειριοποίηση (false serialization) στους βρόχους συσσώρευσης.
- **STARPU_COMMUTE:** Σχεδιασμένο για μαθηματικά αντιμεταθετικές πράξεις. Τα tasks που ζητούν αντιμεταθετική πρόσβαση εγγραφής στην ίδια λαβή δεν χρειάζεται να είναι αυστηρά σειριοποιημένα. Το runtime επιτρέπεται να τα εκτελέσει με οποιαδήποτε σειρά, αυξάνοντας μαζικά τον διαθέσιμο παραλληλισμό, ενώ παράλληλα διασφαλίζει τη συνέπεια έναντι των τυπικών αναγνώσεων.
- **STARPU_SCRATCH:** Δίνει εντολή στο runtime να δεσμεύσει ένα προσωρινό, μη αρχικοποιημένο buffer στον κόμβο-στόχο αποκλειστικά για τη διάρκεια ζωής του task. Αυτό παρακάμπτει την ανάγκη του χρήστη να διαχειρίζεται προσωρινές λειτουργίες malloc ή cudaMalloc μέσα στο codelet, εξαλείφοντας τις δαπανηρές επιβαρύνσεις κατανομής.

3.3.4 Κατάτμηση Δεδομένων και Ιεραρχικά Φίλτρα

Η πυκνή γραμμική άλγεβρα και οι επιλύτες πεπερασμένων διαφορών βασίζονται σε μεγάλο βαθμό σε αλγόριθμους μπλοκ. Η καταχώρηση ενός μονολιθικού πίνακα 10GB ως ενιαία λαβή είναι υπολογιστικά καταστροφική. Μια μεμονωμένη λειτουργία εγγραφής θα ακύρωνε ολόκληρο τον πίνακα σε όλους τους επιταχυντές.

Το StarPU το επιλύει αυτό μέσω ενός προηγμένου συστήματος ιεραρχικής κατάτμησης που χρησιμοποιεί Φίλτρα (struct starpu_data_filter). Ένα κύριο data handle μπορεί να κατατμηθεί ασύγχρονα σε πολλαπλά θυγατρικά data handle. Για παράδειγμα, η starpu_matrix_filter_block() μπορεί να κόψει έναν πίνακα οριζόντια ή κάθετα, χωρίζοντάς τον σε ένα πλέγμα διακριτών υπο-πλακιδίων. Το πρωτόκολλο συνοχής MSI σέβεται αυτή την ιεραρχία. Τα tasks λειτουργούν πάντα στο πιο λεπτομερές διαθέσιμο τμήμα - τα φύλλα του δέντρου δεδομένων. Όταν ένας υπολογιστικός πυρήνας τροποποιεί ένα θυγατρικό πλακίδιο, μόνο η κατάσταση του συγκεκριμένου μεταβαίνει σε Modified, αφήνοντας τον υπόλοιπο γονικό πίνακα απολύτως έγκυρο. Όταν η κατάτμηση αναιρεθεί μέσω της starpu_data_unpartition(), το StarPU συγκεντρώνει έξυπνα τα διάσπαρτα, τροποποιημένα θυγατρικά μπλοκ και τα γράφει πίσω στον συνολικό χώρο μνήμης του γονικού data handle, αποκαθιστώντας τη συνοχή [12, 16].

3.3.5 Ασύγχρονες Μεταφορές και Προανάγνωση (Prefetching)

Για να επιτευχθεί μέγιστη απόδοση υλικού, οι μεταφορές μνήμης πρέπει να αποκρύπτονται εντελώς πίσω από τον ενεργό υπολογισμό. Επειδή το StarPU διαθέτει ολιστική άποψη του γραφήματος εργασιών, διαθέτει ντετερμινιστική γνώση για το ακριβώς ποια data handles θα απαιτηθούν στο μέλλον. Μόλις ο χρονοπρογραμματιστής αναθέσει ένα μελλοντικό task σε μια PU, το υποσύστημα διαχείρισης εκδίδει αμέσως ένα ασύγχρονο αίτημα προανάγνωσης (`starpu_data_prefetch_on_node`). Μέχρι τη στιγμή που το μελλοντικό task γίνεται ενεργό, η απαιτούμενη μνήμη του είναι ήδη τοπικά εγκατεστημένη, εξουδετερώνοντας πλήρως την καθυστέρηση μνήμης. Υποστηρίζεται επίσης το `STARPU_IDLEFETCH`, μια λειτουργία που βάζει στην ουρά μη κρίσιμες προαναγνώσεις όταν ο δίαυλος υλικού είναι εντελώς αδρανής [12, 16].

3.4 Χρονοπρογραμματισμός Εργασιών

Όπως αναφέραμε και στο Κεφάλαιο 2, ο χρονοπρογραμματισμός εργασιών είναι μια κρίσιμη συνιστώσα για την απόδοση των εφαρμογών σε ετερογενή συστήματα. Το StarPU παρέχει μια ευρεία γκάμα αλγορίθμων χρονοπρογραμματισμού που βασίζονται σε μοντέλα απόδοσης πραγματικού χρόνου (**real-time performance models**).

3.4.1 Μοντέλα Απόδοσης Πραγματικού Χρόνου

Αυτοί οι αλγόριθμοι χρησιμοποιούν εκτεταμένα προφίλ απόδοσης που συλλέγονται δυναμικά για να κάνουν ενημερωμένες αποφάσεις σχετικά με το πού και πότε να εκτελέσουν κάθε task [12, 16].

Παρακάτω στον Πίνακα 3.2 παρουσιάζονται τα κύρια μοντέλα απόδοσης που υποστηρίζει το StarPU, μαζί με μια σύντομη περιγραφή και τις περιπτώσεις χρήσης τους:

Τύπος Μοντέλου	Αλγοριθμικός Μηχανισμός	Βέλτιστες Περιπτώσεις Χρήσης
History-Based	Καταγράφει μέσους χρόνους εκτέλεσης σε βελτιστοποιημένο πίνακα κατακερματισμού, ευρετηριασμένο κατά μέγεθος δεδομένων και μονάδα εκτέλεσης.	Τακτικά φορτία με σταθερές διαστάσεις δεδομένων (π.χ. κατακερματισμένη γραμμική άλγεβρα).

Regression-Based	Προσαρμόζει ιστορικά δεδομένα εκτέλεσης σε μαθηματικές καμπύλες.	Φορτία με πολύ μεταβλητά μεγέθη δεδομένων.
Multiple Regression	Προηγμένη προσαρμογή πολυωνυμικής καμπύλης με πολλαπλές μεταβλητές.	Σύνθετα φορτία που εξαρτώνται από πολλαπλές διαστάσεις.
Energy Models	Παρακολουθεί κατανάλωση ισχύος (σε Joule) αντί για χρόνο εκτέλεσης.	Σενάρια που απαιτούν αυστηρό ανώτατο όριο ισχύος ή χρονοπρογραμματισμό με επίγνωση ενέργειας.

Πίνακας 3.2: Μοντέλα Απόδοσης στο StarPU

Εάν το σύστημα συναντήσει εργασία χωρίς ιστορικό στο μοντέλο απόδοσης, εκκινεί δυναμικά φάση βαθμονόμησης — εκτελεί αναγκαστικά την εργασία, τη χρονομετρεί και καταγράφει τη μετρική. Αυτά τα μοντέλα σειριοποιούνται και αποθηκεύονται στο δίσκο (συνήθως στο `.starpup/performmodels`), βελτιώνοντας οργανικά την εξισορρόπηση φορτίου με την πάροδο του χρόνου. Οι προγραμματιστές μπορούν να ελέγξουν αυτή τη συμπεριφορά με μεταβλητές όπως η `STARPU_CALIBRATE=1` ή `STARPU_CALIBRATE=2`. Εργαλεία όπως το `starpup_performmodel_display` επιτρέπουν την οπτικοποίηση αυτών των μετρικών εκτός σύνδεσης, ώστε να μπορούν διαπιστώσουν ότι το υλικό τρέχει όπως προβλέπεται [12, 16, 21].

3.4.2 Πολτικές Χρονοπρογραμματισμού Εργασιών

Οι παραπάνω μετρικές απόδοσης τροφοδοτούν μια ποικιλία πολιτικών χρονοπρογραμματισμού που καθορίζουν πώς το StarPU αντιστοιχίζει τα tasks στους διαθέσιμους workers. Μερικές από τις πιο σημαντικές πολιτικές περιλαμβάνουν [12, 15]:

- **Heterogeneous Earliest Finish Time - HEFT:** Το StarPU υλοποιεί μια δυναμική παραλλαγή του αλγορίθμου HEFT βασισμένο από την θεωρία στατικού χρονοπρογραμματισμού. Όταν ένα task μεταβαίνει στην κατάσταση "Ready", ο HEFT το αξιολογεί ως προς κάθε διαθέσιμο worker. Υπολογίζει τον αναμενόμενο χρόνο εκτέλεσης χρησιμοποιώντας το μοντέλο απόδοσης που έχει ορισθεί. Υπολογίζει επίσης την προβλεπόμενη ποινή μεταφοράς δεδομένων, αξιολογώντας την τρέχουσα θέση των δεδομένων σε σχέση με τον κόμβο μνήμης του worker-στόχου. Στη συνέχεια, το HEFT αναθέτει το task στον worker που εγγυάται τον απολύτως ελάχιστο χρόνο ολοκλήρωσης. Αυτή η προσέγγιση εξισορροπεί επιθετικά την υπολογιστική ισχύ έναντι της καθυστέρησης μετακίνησης δεδομένων [3, 12, 17].

- **Deque Model Data Aware - DMDA:** Ο αλγόριθμος χρονοπρογραμματισμού DMDA βασίζεται στις αρχές του HEFT, αλλά βελτιστοποιείται για εξαιρετικά παράλληλα συστήματα. Διατηρεί ατομικές ουρές εργασιών για κάθε worker. Ταξινομεί ενεργά αυτές τις ουρές με βάση την προτεραιότητα και το κόστος μεταφοράς δεδομένων. Το DMDA μεροληπτεί σε μεγάλο βαθμό (biases) την τοποθέτηση των tasks προς τους workers των οποίων οι κόμβοι μνήμης διαθέτουν ήδη τα απαιτούμενα δεδομένα (Shared ή Modified). Μειώνοντας τις μετακινήσεις, ελαττώνει τον ανταγωνισμό του διαύλου (bus) και εξασφαλίζει υψηλότερο συνολικό κορεσμό των επεξεργαστών. Παραλλαγές όπως τα DMDAS και DMDAP ενσωματώνουν αυστηρή ταξινόμηση ανάλογα με την προτεραιότητα ή την ανάλυση κρίσιμης διαδρομής (critical path). Για παράδειγμα, σε μια παραγοντοποίηση Cholesky, χωρίς προτεραιοποίηση, θα μπορούσε να οδηγήσει σε λιμοκτονία τα κρίσιμα tasks. Το DMDAP διασφαλίζει ότι προωθούνται στο μπροστινό μέρος των ουρών [12, 14, 17].

Επίσης, το StarPU υποστηρίζει και πολιτικές χρονοπρογραμματισμού που δεν βασίζονται σε μοντέλα απόσδοσης. Μερικές από αυτές είναι οι εξής [12]:

- **Eager:** Αυτός ο αλγόριθμος χρονοπρογραμματισμού χρησιμοποιεί μια κεντρική ουρά εργασιών (task queue), από την οποία όλοι οι workers αντλούν εργασίες για ταυτόχρονη εκτέλεση. Αυτό, όμως, δεν επιτρέπει την προανάγνωση (prefetching) δεδομένων, επειδή η απόφαση χρονοπρογραμματισμού λαμβάνεται αργά. Αν μια εργασία έχει προτεραιότητα διαφορετική από το 0, τοποθετείται στην αρχή της ουράς.
- **Random:** Αυτός ο αλγόριθμος χρονοπρογραμματισμού κάνει χρήση μιας ουράς εργασιών για κάθε worker και κατανέμει τυχαία τις εργασίες σε αυτούς με βάση την υποτιθέμενη γενική επίδοση του καθενός.
- **Work Stealing:** Αυτός ο αλγόριθμος χρονοπρογραμματισμού χρησιμοποιεί μια ουρά εργασιών για κάθε worker. Όταν ένας worker εξαντληθεί από εργασίες, είναι ανενεργός (idle), μπορεί να "κλέψει" εργασίες από τις ουρές άλλων workers, επιτρέποντας έτσι την δυναμική εξισορρόπηση φορτίου.
- **Heteroprio:** Αυτός ο αλγόριθμος χρονοπρογραμματισμού αναθέτει για κάθε διαφορετικό PU μια διαφορετική προτεραιότητα και κατανέμει τον φόρτο εργασίας ανάλογα με αυτήν. Απλώς αυτός ο αλγόριθμος θέλει σωστή ρύθμιση για να δουλέψει σωστά με βάση το documentation.

3.5 Κύκλος Εκτέλεσης Εσωτερικών Εργατών

Ο συνεχής κύκλος εκτέλεσης ενός νήματος worker είναι ένας εξαιρετικά βελτιστοποιημένος βρόχος ανάκτησης, εκτέλεσης και συγχρονισμού, σχεδιασμένος να μεγιστοποιεί την απόδοση του υλικού ενώ αποτρέπει την "λιμοκτονία" (starvation) των νημάτων και τα "αδιέξοδα" (deadlocks) σε περιβάλλοντα με υψηλή ταυτοχρονία (concurrency). Σε αντίθεση με έναν απλό βρόχο συμβάντων (event loop) που επεξεργάζεται ασύγχρονα μηνύματα, ο worker του StarPU λειτουργεί ως μια εξαιρετικά εξειδικευμένη μηχανή καταστάσεων.

3.5.1 Ανάκτηση Εργασιών και Διαχείριση Καταστάσεων

Όταν ένας worker είναι έτοιμος να λάβει κάποια εργασία, ρωτάει τον χρονοπρογραμματιστή που είναι συσχετισμένος με το scheduling context του. Κατά τη διάρκεια αυτής της διαδικασίας, ο worker μπαίνει σε μια κρίσιμη φάση που καθορίζεται από την συνάρτηση, `starpu_worker_enter_sched_op()`, όπου αλλάζει το εσωτερικό του flag, `state_sched_op_pending`, σε 0. Αυτή η κατάσταση σηματοδοτεί ότι ο worker αξιολογεί ενεργά την λίστα εργασιών (task queue), υπολογίζει εκτιμήσεις μοντέλων απόδοσης ή προσπαθεί να "κλέψει" δουλειά από άλλους workers [16].

Αυτό το flag κατάστασης λειτουργεί ως αυστηρό μέτρο ασφαλείας για την ταυτόχρονη εκτέλεση. Όσο είναι ενεργό το `state_sched_op_pending`, ο εργαζόμενος προστατεύεται από διαχειριστική παρέμβαση (administrative interference). Άλλα νήματα απαγορεύεται αυστηρά να αφαιρούν τον worker από το scheduling context του ή να αλλάζουν τις αναθέσεις της παράλληλης ομάδας του κατά τη διάρκεια αυτού του διαστήματος [16].

Εάν ένα εξωτερικό νήμα χρειαστεί να τροποποιήσει το πλαίσιο του worker, θέτει μια σημαία ειδοποίησης (`state_changing_ctx_notice = 10`). Αυτό σηματοδοτεί στον worker ότι δεν πρέπει να ξεκινήσει μια νέα λειτουργία χρονοπρογραμματισμού μετά την ολοκλήρωση της τρέχουσας, εμποδίζοντας το διαχειριστικό νήμα να υποστεί "λιμοκτονία". Εάν ο worker είναι ήδη βαθιά ενσωματωμένος σε μια λειτουργία χρονοπρογραμματισμού όταν φτάσει η ειδοποίηση, τίθεται μια δευτερεύουσα σημαία (`state_changing_ctx_waiting = 10`), η οποία δίνει εντολή στον worker να «ξυπνήσει» ενεργά το μπλοκαρισμένο διαχειριστικό νήμα μόλις καλέσει τη συνάρτηση `_starpu_worker_leave_sched_op` [16].

3.5.2 Χαλαρός Συγχρονισμός και Αποφυγή Αδιεξόδων

Μια βασική αρχιτεκτονική ιδέα στο σχεδιασμό του StarPU είναι η προσέγγισή του που βασίζεται στον «χαλαρό συγχρονισμό» (relaxed synchronization). Οι προηγμένοι αλγόριθμοι (όπως το Work Stealing) απαιτούν από τους workers να παρατηρούν το φορτίο, το μήκος της ουράς και την κατάσταση των ομότιμων workers τους για να λάβουν βέλτιστες

αποφάσεις εξισορρόπησης φόρτου. Εάν το StarPU χρησιμοποιούσε αυστηρό κλείδωμα mutex ¹ (π.χ. ο Worker A κλειδώνει το sched_mutex του Worker B), αναπόφευκτα θα προκαλούσε αδιέξοδα (deadlocks) σε ένα αποκεντρωμένο σύστημα (ο Worker A κλειδώνει τον B, ενώ ο B προσπαθεί να κλειδώσει τον A) [16].

Ο χαλαρός συγχρονισμός επιτρέπει στους workers να διαβάζουν με ασφάλεια τα ατομικά flags κατάστασης των ομοτίμων τους χωρίς να ξεκινούν ένα μπλοκάρισμα κλειδώματος. Η αναζήτηση πληροφοριών κατάστασης επιτρέπεται εάν `state_relax_refcnt > 0`, επιτρέποντας εξαιρετικά κλιμακούμενους αλγόριθμους κλοπής εργασίας και εξισορρόπησης φόρτου χωρίς κλειδώματα (lock-free) σε δεκάδες διακριτούς πυρήνες, χωρίς να σταματά η διοχέτευση εκτέλεσης. Μόλις ανακτηθεί επιτυχώς ένα task, ο worker χρησιμοποιεί τον οδηγό (driver) του για να ζητήσει από το υποσύστημα Εικονικής Κοινόχρηστης Μνήμης (VSM) να φέρει τα απαραίτητα data handles στην τοπική μνήμη. Μετά την επίλυση των μεταφορών μνήμης, ο worker καλεί τον κατάλληλο δείκτη συνάρτησης του codelet, εκτελεί τον μαθηματικό πυρήνα και επικαλείται τυχόν καταχωρημένες συναρτήσεις επανάκλησης προτού επιστρέψει στην αρχή του βρόχου [16].

3.6 Κατανεμημένη Εκτέλεση: MPI

Παρόλο που το StarPU βελτιστοποιεί την αξιοποίηση του υλικού ενός κόμβου με διάφορες ετερογενείς μονάδες επεξεργασίας (PU), η ανάγκη για περεταίρω κλιμάκωση σε πολλαπλούς κόμβους είναι εξίσου σημαντική στις σημερινές εφαρμογές HPC. Για να υποστηρίξει αυτό το σενάριο, το StarPU ενσωματώνει ένα υποσύστημα κατανεμημένης εκτέλεσης βασισμένο στο MPI (Message Passing Interface), μετατρέποντας ένα μεγάλο cluster σε ένα ενιαίο ετερογενές σύστημα [12, 16].

3.6.1 Υποβολή Εργασιών σε Κατανεμημένο Περιβάλλον

Ιστορικά, ο κατανεμημένος προγραμματισμός απαιτούσε τη χειροκίνητη παρεμβολή ρουτινών MPI_Send και MPI_Recv. Το StarPU εξαλείφει πλήρως αυτό το παράδειγμα μέσω του `starpu_mpi_task_insert`. Ο προγραμματιστής καταχωρεί τα data handles με έναν συγκεκριμένο βαθμό (rank) MPI, χρησιμοποιώντας τη συνάρτηση `starpu_mpi_data_register`. Αυτό καθιερώνει την έννοια της «ιδιοκτησίας» για τα εν λόγω δεδομένα. Κατά την εκτέλεση, κάθε διεργασία MPI στο cluster εκτελεί τον ίδιο ακριβώς κώδικα εφαρμογής, ξεδιπλώνοντας το ίδιο ακριβώς καθολικό γράφημα εργασιών [12].

Όταν καλείται το `starpu_mpi_task_insert`, η μηχανή StarPU-MPI εκτελεί έναν ντετερμινιστικό αλγόριθμο συναίνεσης:

¹Ένας **mutex** (από το mutual exclusion, αμοιβαίος αποκλεισμός) είναι ένας μηχανισμός συγχρονισμού στην επιστήμη των υπολογιστών που χρησιμοποιείται για την προστασία κοινόχρηστων πόρων.

1. **Ανάθεση Κόμβου Εκτέλεσης:** Ο κόμβος MPI που κατέχει το data handle σε λειτουργία STARPU_W ή STARPU_RW επιλέγεται ως ο εκτελεστής του task. Μπορεί να παρακαμφθεί μέσω του STARPU_EXECUTE_ON_NODE.
2. **Έμμεση Δρομολόγηση Δεδομένων (Send):** Εάν ένας κόμβος MPI κατέχει δεδομένα που απαιτούνται (Ανάγνωση), αλλά δεν είναι ο κόμβος εκτέλεσης, το StarPU δημοσιεύει αυτόματα ένα ασύγχρονο MPI_Isend για τη μετάδοση.
3. **Έμμεση Δρομολόγηση Δεδομένων (Receive):** Ο κόμβος εκτέλεσης δημοσιεύει αυτόματα MPI_Irecv για τη συλλογή των δεδομένων Ανάγνωσης.
4. **Εκτέλεση και Επαναγραφή (Write-back):** Ο κόμβος εκτέλεσης λαμβάνει τα δεδομένα, επεξεργάζεται το codelet και τροποποιεί το τοπικό data handle Εγγραφής. Εάν τα δεδομένα πρέπει να ωθηθούν πίσω σε έναν ιδιοκτήτη, το StarPU χειρίζεται το MPI_Isend αυτόματα.

Το καινοτόμο με τον παραπάνω μηχανισμό είναι ότι ο βασικός αλγοριθμικός κώδικας δεν απαιτεί απολύτως καμία δήλωση επικοινωνίας MPI. Ο ίδιος ακριβώς σειριακός κώδικας κλιμακώνεται απρόσκοπτα σε χιλιάδες κόμβους [12].

3.6.2 Βελτιστοποιήσεις Επικοινωνίας MPI

Για να αποτραπεί η καθυστέρηση της CPU, το StarPU αφιερώνει ένα αποκλειστικό, καρφίτωμένο (pinned) νήμα CPU αποκλειστικά στη μηχανή προόδου του MPI. Αυτό το νήμα παρασκηνίου ελέγχει συνεχώς το δίκτυο, εκδίδοντας κλήσεις MPI_Test ασύγχρονα χωρίς να επιρεάζει του τοπικούς εργάτες (workers). Επιπλέον, για την ελαχιστοποίηση της συμφόρησης του δικτύου, το StarPU-MPI υλοποιεί μια έξυπνη κρυφή μνήμη επικοινωνίας (starpu_mpi_cache_set) [12].

3.7 Σημαντικά Υποσυστήματα του StarPU

Το StarPU πέρα από όλα τα βασικά συστήματα που αναλύσαμε στις προηγούμενες ενότητες, διαθέτει και άλλα σημαντικά υποσυστήματα που συμβάλλουν στην απόδοση και την ευελιξία του. Μερικά από αυτά είναι θα τα δούμε σύντομα παρακάτω.

3.7.1 Out-of-Core Εκτέλεση

Όταν το μέγεθος του προβλήματος υπερβαίνει τη διαθέσιμη κύρια RAM, το σύστημα παραδοσιακά θα κατέρρεε. Το StarPU το λύνει αυτό ενσωματώνοντας μονάδες δίσκου υψηλής ταχύτητας (NVMe) στην ιεραρχία Εικονικής Κοινόχρηστης Μνήμης (VSM) ως

διακριτούς Κόμβους Μνήμης Δίσκου. Η πολιτική **LRU** προωθεί ασύγχρονα τα data handles απευθείας στον δίσκο. Όταν ζητηθούν ξανά, το runtime τα φέρνει πίσω στη RAM ή ξεκινά μια μεταφορά Άμεσης Πρόσβασης Μνήμης (**DMA**) απευθείας από τον αποθηκευτικό χώρο στη μνήμη της PU, παρακάμπτοντας εντελώς τη CPU [12].

3.7.2 Context Hypervisors και Παράλληλες Εργασίες

Για εξαιρετικά ακανόνιστους (irregular) φόρτους εργασίας, η στατική κατανομή workers μπορεί να οδηγήσει σε υποαπασχόληση του υλικού. Το StarPU εισάγει τον Scheduling Context Hypervisor, ο οποίος αναλύει τους χρόνους αδράνειας, τις ταχύτητες εκτέλεσης και τα μήκη των ουρών σε πραγματικό χρόνο. Χρησιμοποιώντας όρια γραμμικού προγραμματισμού, ο hypervisor αλλάζει αυτόματα το μέγεθος των πλαισίων, μεταφέροντας αδρανείς workers CPU, GPU ή γενικά PU από ένα εξαντλημένο πλαίσιο σε ένα κορεσμένο. Το StarPU υποστηρίζει εγγενώς παράλληλα tasks μέσω OpenMP (starpu_omp_init). Ένα ενιαίο **Codelet** μπορεί να ενσωματώσει μια παράλληλη περιοχή του OpenMP. Το StarPU θα αναθέσει το Job σε μια ολόκληρη ομάδα workers της CPU ταυτόχρονα, με το σωστό καρφίτσωμα πυρήνα και επίγνωση της αρχιτεκτονικής **NUMA** [12].

3.7.3 Ανοχή Σφαλμάτων (Fault Tolerance)

Σε τεράστια κατανεμημένα υπολογιστικά συστήματα, οι αστοχίες κόμβων είναι στατιστικά αναπόφευκτες. Το StarPU παρέχει ενσωματωμένη υποστήριξη μέσω λήψης σημείων ελέγχου (checkpointing) (starpu_mpi_checkpoint_template_register). Το runtime επιτρέπει τον ορισμό προτύπων σημείων ελέγχου που «παγώνουν» (freeze) την κατάσταση συγκεκριμένων data handles. Εάν ένας κόμβος αποτύχει καταστροφικά, η εφαρμογή μπορεί να επιστρέψει (roll back) σε ένα σημείο ελέγχου και να επαναπρογραμματίσει δυναμικά το χαμένο γράφημα εργασιών, εξασφαλίζοντας ότι μεγάλης κλίμακας προσομοιώσεις επιβιώνουν από τη φθορά του υλικού [12].

Κεφάλαιο 4

Η Διεπαφή StarPUPy

4.1 Τι είναι το StarPUPy

Το **StarPUPy** είναι μια διεπαφή προγραμματισμού στην γλώσσα υψηλού επιπέδου **Python** και μεταφέρει τις δυνατότητες του StarPU σε ένα φιλικό προς τον χρήστη περιβάλλον. Στόχος του είναι να επιτρέψει στους προγραμματιστές να εκμεταλλευτούν τις δυνατότητες του StarPU σε συνδυασμό με την ευκολία, την απλότητα και το πλούσιο οικοσύστημα της Python.

Ως επί το πλείστον, λειτουργεί ως ένα wrapper γύρω από τις βασικές λειτουργίες του StarPU στην χαμηλού επιπέδου γλώσσα προγραμματισμού **C**, επιτρέποντας στους επιστήμονες δεδομένων που χρησιμοποιούν αποκλειστικά Python να αξιοποιήσουν τη δύναμη του ετερογενούς χρονοπρογραμματισμού του C backend, διευκολύνοντας την ενσωμάτωση με βιβλιοθήκες όπως το **NumPy** και το **SciPy**, οι οποίες κυριαρχούν στην έρευνα [12].

4.2 Ασύγχρονη Υποβολή Εργασιών και Διαχείριση Αντικειμένων Future

Η υποβολή εργασιών στο StarPUPy βασίζεται στη χρήση της συνάρτησης `task_submit(options)(func, *args, **kwargs)`. Η κλήση αυτής της συνάρτησης δεν εκτελείται άμεσα, αλλά επιστρέφει ένα αντικείμενο **Future** της βιβλιοθήκης **asyncio** της Python. Τα αντικείμενα **Future** αποτελούν **awaitable** οντότητες που αντιπροσωπεύουν την μελλοντική ολοκλήρωση μιας ασύγχρονης εργασίας. Αυτό επιτρέπει στους προγραμματιστές να υποβάλλουν εργασίες και να περιμένουν τα αποτελέσματα από αυτές χωρίς να μπλοκάρει το βασικό νήμα (**main thread**) εκτέλεσης [12].

Στον Πίνακα 4.1 παρουσιάζονται τα βασικά στοιχεία της συνάρτησης `task_submit` και των παραμέτρων της [12]:

Επιλογή (Option)	Τύπος Δεδομένων	Προεπιλεγμένη Τιμή Χρήσης	Λειτουργική Επίδραση
name	string	None	Ορισμός αναγνωριστικού ονόματος της εργασίας για χρήση σε εργαλεία αποσφαλμάτωσης.
synchronous	unsigned	0	Εάν οριστεί σε 1, η υποβολή μπλοκάρει την εκτέλεση μέχρι την ολοκλήρωση της εργασίας.
priority	int	0	Καθορίζει το επίπεδο προτεραιότητας της εργασίας εντός του εύρους τιμών των sched_get_min_priority() και sched_get_max_priority().
color	unsigned	None	Ορισμός χρώματος της εργασίας για την οπτικοποίηση του γραφήματος εξαρτήσεων (DAG) στο εργαλείο dag.dot.
flops	double	None	Δηλώνει το υπολογιστικό μέγεθος σε πράξεις κινητής υποδιαστολής για τη δημιουργία καμπυλών απόδοσης.
perfmodel	string	None	Καθορίζει το όνομα του μοντέλου επίδοσης για την ιστορική βαθμονομημένη χρονοδρομολόγηση.

Πίνακας 4.1: Επιλογές για την συνάρτηση task_submit του StarPUPy

Η Python 3.8+ υποστηρίζει την εκτέλεση με την παράμετρο `-m asyncio`, η οποία επιτρέπει την άμεση χρήση της εντολής `await` στο διαδραστικό περιβάλλον (REPL ¹). Επιπλέον, παρέχεται ο διακοσμητής `@starpupy.delayed`, ο οποίος μετατρέπει μια τυπική συνάρτηση σε ασύγχρονη εργασία StarPU, δεχόμενος τις ίδιες παραμέτρους χρονοδρομολόγησης [12].

4.3 Διαχείριση Μνήμης και Πρωτόκολλο Buffer

Ένα από τα σημαντικότερα χαρακτηριστικά του StarPUPy είναι η ενσωμάτωση του πρωτοκόλλου buffer (buffer protocol) της Python. Το πρωτόκολλο αυτό επιτρέπει στο StarPUPy να αποκτά άμεση πρόσβαση στα ακατέργαστα byte arrays αντικειμένων, όπως οι πίνακες NumPy, χωρίς την ανάγκη αντιγραφής ή σειριοποίησης.

Το σύστημα διατήρησης της συνοχής μνήμης του StarPU απαιτεί τη δήλωση του τρόπου πρόσβασης σε κάθε μεταβλητή. Αυτό επιτυγχάνεται με τη χρήση του διακοσμητή

¹Το **REPL (Read-Eval-Print Loop)** είναι ένα διαδραστικό περιβάλλον προγραμματισμού όπου εισάγετε μια εντολή (Read), ο υπολογιστής την εκτελεί άμεσα (Eval) και εμφανίζει το αποτέλεσμα (Print), επαναλαμβάνοντας τη διαδικασία (Loop) για την επόμενη εντολή.

@starpu.access:

- **R (Read):** Ανάγνωση δεδομένων (προεπιλογή).
- **W (Write):** Αποκλειστική εγγραφή δεδομένων.
- **RW (Read-Write):** Ανάγνωση και τροποποίηση των δεδομένων.

Όταν ένα αντικείμενο υποβάλλεται σε μια εργασία με δικαιώματα εγγραφής (W ή RW), η εφαρμογή Python δεν μπορεί να προσπελάσει τις τιμές του άμεσα, καθώς αυτές ενδέχεται να βρίσκονται σε μια διακριτή μνήμη PU. Για την προσπέλαση εκτός των εργασιών StarPU, χρησιμοποιείται το σχήμα Acquire/Release. Εάν η εφαρμογή επιχειρήσει να καταχωρίσει το ίδιο μεταβαλλόμενο (mutable) αντικείμενο Python περισσότερες από μία φορές στο σύστημα διαχείρισης δεδομένων, το StarPUPy εγείρει μια ρητή εξαίρεση ασφαλείας, *"starpu.error: Should not register the same mutable python object once more."* [12].

Επιπλέον, οι πίνακες NumPy πολλών διαστάσεων μπορούν να υποστούν κατάτμηση σε Segment Chunks για παράλληλη επεξεργασία μέσω της μεθόδου `Handle::partition(nchildren, dim, chunks_list)`. Η φυσική διεύθυνση μνήμης και το μέγεθος σε byte για κάθε υπο-πίνακα υπολογίζονται αυτόματα από το StarPUPy. Μετά την ολοκλήρωση των παράλληλων υπολογισμών στα sub-handles, η κλήση της `Handle::unpartition(handle_list, nchildren)` ενσωματώνει τα αποτελέσματα στον αρχικό πίνακα, διασφαλίζοντας τη συνοχή των δεδομένων στην κύρια μνήμη RAM [12].

Τέλος, το StarPUPy υποστηρίζει την χρήση data handles όπως και το StarPU C API, επιτρέποντας την εγγραφή σε δεδομένα που βρίσκονται σε διακριτή μνήμη PU. Αυτό επιτυγχάνεται με τη χρήση της κλάσης `Handle`¹ και την δημιουργία ενός αντικειμένου πάνω στα επιθυμητά δεδομένα. Μετά την εγγραφή, το αντικείμενο μπορεί να χρησιμοποιηθεί σε εργασίες StarPU με τα κατάλληλα δικαιώματα πρόσβασης [12].

4.4 Παραλληλία και Αντιμετώπιση του GIL

Η επίτευξη πραγματικής παραλληλίας σε περιβάλλον Python περιορίζεται παραδοσιακά από το **Global Interpreter Lock (GIL)**, το οποίο επιβάλλει τη σειριακή εκτέλεση του πηγαίου κώδικα Python. Το StarPUPy αντιμετωπίζει αυτή την πρόκληση μέσω διάφορων στρατηγικών [12]:

- **Αποδέσμευση GIL μέσω C-APIs:** Κατά την κλήση εξωτερικών βιβλιοθηκών (π.χ. πράξεις γραμμικής άλγεβρας του NumPy που βασίζονται σε βιβλιοθήκες BLAS), το

¹Η κλάση `Handle` είναι μέρος του module `starpu`.

GIL απελευθερώνεται αυτόματα. Αυτό επιτρέπει στους εργάτες CPU του StarPU να εκτελούν τις εργασίες ταυτόχρονα.

- **Multi-interpreter (Python 3.12+):** Η νεότερη αρχιτεκτονική της Python επιτρέπει την ύπαρξη ενός ανεξάρτητου GIL ανά διεργασία (PEP 0684). Το StarPUPy υποστηρίζει πλήρως αυτή τη λειτουργία μέσω της μεταβλητής `STARPU_PYTHON_OWN_GIL=1`, αν και ορισμένες λειτουργίες, όπως τα `asyncio futures`, παρουσιάζουν περιορισμούς συμβατότητας στην τρέχουσα έκδοση.
- **MPI - Master-Slave:** Σε περιβάλλοντα με πολλαπλούς κόμβους, το StarPUPy μπορεί να χρησιμοποιήσει το **Message Passing Interface (MPI)**, που είναι υποστηριζόμενο από το backend στην γλώσσα C, για την επικοινωνία μεταξύ διεργασιών. Σε αυτό το σενάριο, ο κύριος κόμβος (master) διαχειρίζεται την υποβολή εργασιών και τη συλλογή αποτελεσμάτων, ενώ οι δούλοι (slaves) εκτελούν τις εργασίες ανεξάρτητα, παρακάμπτοντας το GIL.

Αν και οι τελευταίες δύο στρατηγικές είναι υποσχόμενες δεν υποστηρίζονται και δεν τεκμηριώνονται πλήρως στην τρέχουσα έκδοση του StarPUPy, και η υποστήριξη για αυτές τις λειτουργίες αναμένεται να βελτιωθεί σε μελλοντικές εκδόσεις.

4.5 Παράδειγμα Χρήσης του StarPUPy

Στην παρακάτω ενότητα παρουσιάζεται ένα απλό παράδειγμα χρήσης του StarPUPy, το οποίο καλύπτει την υποβολή εργασιών και την γενική φιλοσοφία της διεπαφής με βάση ότι έχουμε αναφέρει μέχρι τώρα.

Στο παράδειγμα αυτό δημιουργούνται και υποβάλλονται 8 CPU-bound εργασίες (busy loop) μέσω του StarPUPy. Η εκτέλεση των εργασιών μπορεί να παρακολουθηθεί σε πραγματικό χρόνο με την χρήση του εργαλείου `htop` σε linux σύστημα, το οποίο εμφανίζει τα νήματα/διεργασίες και τη χρήση της CPU.

Θεωρούμε ότι το StarPUPy έχει εγκατασταθεί σωστά και είναι διαθέσιμο στο περιβάλλον Python. Οδηγίες γι' αυτό μπορείτε να βρείτε εδώ [12].

Αρχικά πρέπει να εισάγουμε το module `starpu` μαζί με τις απαραίτητες βιβλιοθήκες και να ορίσουμε μια συνάρτηση που θα εκτελεί την CPU-bound εργασία. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```
import time
import asyncio
import os
```

```

from starpu import starpu

def busy_work(seconds):
    # CPU-bound busy loop to generate measurable load
    t0 = time.time()
    x = 1
    while time.time() - t0 < seconds:
        # cheap integer ops to keep CPU busy
        x = ((x << 1) ^ (x >> 3)) & 0xFFFFFFFF
    return x

```

Για την δημιουργία και υποβολή των εργασιών (tasks), που το καθένα θα εκτελεί την παραπάνω συνάρτηση `busy_work()`, χρησιμοποιούμε μια ασύγχρονη συνάρτηση, `as` την πούμε `main()`, η οποία πρώτα αρχικοποιεί το StarPUPy περιβάλλον καλώντας την συνάρτηση `starpu.init()`. Στην συνέχεια μέσα σε ένα βρόχο `for` δημιουργούνται και υποβάλλονται 8 εργασίες, οι οποίες εκτελούν την συνάρτηση `busy_work()` για 8 δευτερόλεπτα η κάθε μία. Αυτό γίνεται με την επανειλημμένη χρήση της συνάρτησης `starpu.task_submit()`. Τα αντικείμενα `Future` που επιστρέφονται από την υποβολή των εργασιών αποθηκεύονται σε μια λίστα `futures`. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```

async def main():
    try:
        starpu.init()
    except Exception as e:
        print(f"StarPU initialization error: {e}")
        return

    print(f"PID: {os.getpid()}, ntasks=8, sec=8.0")
    print("Start submitting tasks...")

    t0 = time.time()
    futures = []
    for i in range(8):
        fut = starpu.task_submit()(busy_work, 8.0)
        futures.append(fut)

```

Στην συνέχεια, αφού έχουν υποβληθεί όλες οι εργασίες, η `main()` συνάρτηση περιμένει την ολοκλήρωση τους και συλλέγει τα αποτελέσματα από τα αντικείμενα `Future`. Αυτό

γίνεται με την χρήση ενός δεύτερου βρόχου for, όπου για κάθε αντικείμενο Future καλείται η εντολή `await fut`, η οποία μπλοκάρει την εκτέλεση μέχρι να ολοκληρωθεί η αντίστοιχη εργασία. Τα αποτελέσματα αποθηκεύονται σε μια λίστα `results`. Μετά την ολοκλήρωση όλων των εργασιών, εμφανίζονται στην οθόνη ο χρόνος εκτέλεσης και τερματίζεται το περιβάλλον StarPUPy με την κλήση της συνάρτησης `starpup.shutdown()`. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```
# Wait for completion and collect results (force materialization)
results = []
for fut in futures:
    res = await fut
    results.append(res)

t1 = time.time()
elapsed = t1 - t0

print("=" * 60)
print("Framework: StarPU Python (Monitor dummy workload)")
print(f"Submitted tasks: 8")
print(f"Work per task (s): 8.0")
print(f"Elapsed time: {elapsed:.4f} seconds")
print("=" * 60)

starpup.shutdown()
```

Τέλος για την εκτέλεση της `main()` συνάρτησης χρησιμοποιούμε την εντολή `asyncio.run(main())`, η οποία εκκινεί το ασύγχρονο περιβάλλον και περιμένει την ολοκλήρωση της `main()` συνάρτησης. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```
if __name__ == '__main__':
    try:
        asyncio.run(main())
    except Exception as e:
        print(f"Error: {e}")
```

4.5.1 Ολοκληρωμένος Κώδικας Παραδείγματος

Παρακάτω παρουσιάζεται ο ολοκληρωμένος κώδικας του παραδείγματος χρήσης του StarPUPy, ο οποίος περιλαμβάνει όλα τα προηγούμενα αποσπάσματα κώδικα:

```

"""
Monitor example for StarPU Python
- Submits multiple CPU-bound dummy tasks (busy loop) via StarPU
- Use `htop` while it runs to observe threads/processes and CPU usage
"""

import time
import asyncio
import os
from starpu import starpu

def busy_work(seconds):
    # CPU-bound busy loop to generate measurable load
    t0 = time.time()
    x = 1
    while time.time() - t0 < seconds:
        # cheap integer ops to keep CPU busy
        x = ((x << 1) ^ (x >> 3)) & 0xFFFFFFFF
    return x

async def main():
    try:
        starpu.init()
    except Exception as e:
        print(f"StarPU initialization error: {e}")
        return

    print(f"PID: {os.getpid()}, ntasks=8, sec=8.0")
    print("Start submitting tasks...")

    t0 = time.time()
    futures = []
    for i in range(8):
        fut = starpu.task_submit()(busy_work, 8.0)
        futures.append(fut)

    # Wait for completion and collect results (force materialization)
    results = []
    for fut in futures:
        res = await fut
        results.append(res)

```

```
t1 = time.time()
elapsed = t1 - t0

print("=" * 60)
print("Framework: StarPU Python (Monitor dummy workload)")
print(f"Submitted tasks: 8")
print(f"Work per task (s): 8.0")
print(f"Elapsed time: {elapsed:.4f} seconds")
print("=" * 60)

starpu.shutdown()

if __name__ == '__main__':
    try:
        asyncio.run(main())
    except Exception as e:
        print(f"Error: {e}")
```

Κεφάλαιο 5

Το Σύστημα torcpsy

5.1 Τι είναι το torcpsy

Πέρα από την ανάγκη για λύσεις για την πλήρη αξιοποίηση των ετερογενών πόρων ενός υπολογιστικού κόμβου σε εφαρμογές **HPC** (π.χ. StarPU), υπάρχει και η ανάγκη για λύσεις που μεγιστοποιούν την απόδοση σε μία συστάδα (cluster) υπολογιστικών κόμβων. Μια προσέγγιση για την επίτευξη αυτού του σκοπού είναι η χρήση του **MPI** (Message Passing Interface) για την επικοινωνία μεταξύ των κόμβων. Ωστόσο, η χρήση του **MPI** μπορεί να είναι περίπλοκη και απαιτεί σημαντική προσπάθεια για την ανάπτυξη και τη συντήρηση των εφαρμογών.

Ταυτόχρονα, η συνεχής και ταχεία εξέλιξη και προσθήκη νέων βιβλιοθηκών κάνουν την γλώσσα **Python** πιο αποτελεσματική για την ανάπτυξη προηγμένων αλγορίθμων και επιστημονικών εφαρμογών. Επιπλέον, προσφέρει διεπαφές υψηλού επιπέδου που διασυνδέουν τους χρήστες με ταχύτατες βιβλιοθήκες HPC γραμμένες σε γλώσσες συστήματος όπως η C, η C++ και η Fortran.

Μία λύση που προσπαθεί να συνδυάσει τα πλεονεκτήματα της **Python** με την απόδοση του **MPI** είναι το **torcpsy**. Το torcpsy δεν σχεδιάστηκε από το μηδέν ως μια απομονωμένη ιδέα, αλλά αποτελεί τη φυσική εξέλιξη και την αμιγή υλοποίηση σε Python (pure Python implementation) της ισχυρής βιβλιοθήκης χρόνου εκτέλεσης TORC (Task Orchestration and Routing Code), η οποία ήταν γραμμένη σε C/C++ [2].

Το **torcpsy** είναι ένα σύστημα χρόνου εκτέλεσης ανοιχτού κώδικα, αγνωστικιστικού ως προς την πλατφόρμα (platform-agnostic), το οποίο υλοποιεί προσαρμοστική εξισορρόπηση φόρτου (adaptive load balancing) και ενορχηστρώνει τον χρονοπρογραμματισμό εργασιών τόσο σε συστήματα κοινόχρηστης όσο και κατανεμημένης μνήμης [2].

Το παρόν κεφάλαιο παρουσιάζει αναλυτικά το σύστημα **torcpsy**, εστιάζοντας στις βασικές του αρχές, την αρχιτεκτονική του και τις δυνατότητες που προσφέρει για την επίτευξη των στόχων που θέσαμε ως το βασικό κίνητρο για την αναπτυξή του.

5.2 Αρχιτεκτονική του torcpy

Για την επίτευξη των προαναφερθέντων στόχων, το torcpy υλοποιήθηκε ως μια λεπτή αλλά εξαιρετικά αποδοτική στιβάδα λογισμικού πάνω από την τεχνολογία του MPI (συγκεκριμένα αξιοποιώντας το πακέτο `mpi4py`) και την τεχνολογία του multithreading της Python (αξιοποιώντας τα πακέτα `threading` και `queue`).

Η θεμελιώδης αρχιτεκτονική διάταξη μιας εφαρμογής που βασίζεται στο torcpy εδράζεται στην ανάπτυξη πολλαπλών διεργασιών MPI, οι οποίες κατανέμονται στους διαθέσιμους υπολογιστικούς κόμβους του συστήματος. Στο εσωτερικό κάθε διεργασίας MPI κατοικεί ένα εξαιρετικά βελτιστοποιημένο υποσύστημα νημάτων (thread subsystem) που αποτελείται από τρεις βασικούς πυλώνες [2]:

1. **Νήματα Εργαστών (Worker Threads):** Κάθε διεργασία περιλαμβάνει ένα ή περισσότερα worker threads. Ο αποκλειστικός ρόλος τους είναι να εξάγουν (extract) εργασίες από τις διαθέσιμες ουρές και να εκτελούν τον παραγόμενο κώδικα Python.
2. **Νήμα Εξυπηρετητή (Server Thread):** Αποτελεί την καρδιά του συστήματος επικοινωνίας και τη γέφυρα με τον εξωτερικό κόσμο (άλλες διεργασίες MPI). Κάθε διεργασία διαθέτει ένα εξειδικευμένο νήμα διακομιστή (dedicated server thread), το οποίο διαχειρίζεται αποκλειστικά τις απομακρυσμένες λειτουργίες (remote operations) διαχείρισης εργασιών και δεδομένων. Όλα τα μηνύματα που αφορούν επικοινωνία μεταξύ των κόμβων ανταλλάσσονται μέσω αυτού του νήματος με διαφανή προς τον χρήστη τρόπο.
3. **Ουρές Εργασιών (Task Queues):** Ένα σύνολο από ιεραρχικές ουρές (set of queues) διατηρείται στην τοπική μνήμη της διεργασίας. Οι εργασίες υποβάλλονται σε αυτές τις ουρές, οι οποίες κατηγοριοποιούνται και οργανώνονται σύμφωνα με το επίπεδο παραλληλισμού (level of parallelism) από το οποίο αρχικά δημιουργήθηκαν. Για παράδειγμα, το αρχικό πρόγραμμα δημιουργεί εργασίες στην ουρά "επιπέδου 1". Αν μια τέτοια εργασία γεννήσει η ίδια νέες εργασίες, αυτές θα τοποθετηθούν στην ουρά "επιπέδου 2", και ούτω καθεξής.

Στην περίπτωση όπου ο χρήστης εκτελεί την εφαρμογή ως αμιγώς πολυνηματικό (multithreaded) κώδικα σε ένα μόνο μηχάνημα —δηλαδή δημιουργώντας μόνο μία διεργασία MPI η οποία φιλοξενεί πολλαπλά νήματα εργατών— το torcpy προσαρμόζεται αυτόματα. Σε αυτό το σενάριο, όλες ανεξαιρέτως οι λειτουργίες υποβολής, εξαγωγής και μεταφοράς δεδομένων πραγματοποιούνται αποκλειστικά μέσω της ταχύτατης τοπικής διαμοιραζόμενης μνήμης (shared memory), απαλείφοντας εντελώς το κόστος των κλήσεων του δικτύου MPI [2].

5.2.1 Δομή Εργασιών και Διαχείριση Αποτελεσμάτων

Μια κρίσιμη απόφαση σχεδιασμού για την εξασφάλιση της ταχύτητας του συστήματος αφορά τη δομή των ίδιων των εργασιών. Προκειμένου να αποφευχθεί το μεγάλο υπολογιστικό κόστος (overhead) που συνεπάγεται η δημιουργία, η δέσμευση μνήμης και η σειριοποίηση (serialization) ογκωδών κλάσεων, οι εργασίες υλοποιούνται αποκλειστικά ως δομές λεξικών της Python (Python dictionaries). Τα λεξικά της Python είναι δομές υψηλής ταχύτητας στο επίπεδο της γλώσσας C (cpython backend), μειώνοντας κατακόρυφα τον χρόνο που απαιτείται για τη δημιουργία της εργασίας [2].

Όταν η εργασία ολοκληρώνεται, το αποτέλεσμα της συνάρτησης αποθηκεύεται με απόλυτη διαφάνεια πίσω στον περιγραφέα της εργασίας (task descriptor ή future). Το εντυπωσιακό είναι ότι το αποτέλεσμα επιστρέφεται και αποθηκεύεται απευθείας στη διεργασία MPI η οποία δημιούργησε (spawned) την εργασία αρχικά, λύνοντας το πρόβλημα της διαχείρισης της μνήμης για τον προγραμματιστή. Ακολουθώντας πιστά το πρότυπο PEP 3148, ο τελικός χρήστης μπορεί μετέπειτα να αποκτήσει πρόσβαση στο αποτέλεσμα χρησιμοποιώντας απλώς την κλήση της μεθόδου `task.result()`. Με πανομοιότυπο τρόπο, παρέχεται η δυνατότητα πρόσβασης στα αρχικά ορίσματα της εργασίας μέσω της κλήσης `task.input()` [2].

5.2.2 Ο Ρόλος και η Λειτουργία του Νήματος Εξυπηρετητή (Server Thread)

Το Server Thread αποτελεί τον ενορχηστρωτή του συστήματος και λειτουργεί συνεχώς στο παρασκήνιο. Η λειτουργία του είναι ασύγχρονη και είναι επιφορτισμένο με τρεις κύριες, ζωτικής σημασίας ευθύνες [2]:

1. **Λήψη και Ένταξη:** Αναλαμβάνει τη λήψη νέων, εισερχόμενων εργασιών που καταφθάνουν από άλλους κόμβους (μέσω του δικτύου MPI) και αναλαμβάνει την τοποθέτησή τους (insertion) στις κατάλληλες τοπικές ουρές της διεργασίας του.
2. **Συλλογή Αποτελεσμάτων:** Παραλαμβάνει τις εργασίες που έχουν πλέον ολοκληρωθεί, μαζί με τα αντίστοιχα αποτελέσματά τους, δρομολογώντας τα σωστά στον κώδικα που τα αναμένει.
3. **Εξυπηρέτηση Αιτημάτων Κλοπής (Task Stealing Requests):** Η πιο σύνθετη λειτουργία του είναι ο χειρισμός των αιτημάτων κλοπής που καταφθάνουν από άλλες, αδρανείς διεργασίες MPI που αναζητούν φόρτο εργασίας.

Για να διασφαλιστεί ότι η επικοινωνία στο δίκτυο δεν θα μπλοκάρει την εκτέλεση του υπόλοιπου κώδικα Python (και κυρίως για να αποφευχθεί ο εγκλωβισμός του GIL

της Python από το νήμα του διακομιστή), ο σχεδιασμός επιβάλλει ρητά τη χρήση μη-μπλοκαρισμένων (non-blocking) κλήσεων του MPI. Κατά την ανάπτυξη και τη βελτιστοποίηση του torcpry, η ομάδα διαπίστωσε μέσα από τη χρήση εργαλείων αξιολόγησης από το πακέτο `mpi4py.futures` (όπως οι ρουτίνες `run_julia.py` και `perf_primes.py`) ότι οι μπλοκαρισμένες κλήσεις `MPI_Recv` δημιουργούσαν τεράστια συμφόρηση. Η λύση που υιοθετήθηκε στηρίχθηκε στην ενσωμάτωση non-blocking κλήσεων σε συνδυασμό με έναν μηχανισμό συνειδητής εκχώρησης του επεξεργαστή (`processor yielding`). Όταν το νήμα του διακομιστή δεν έχει μηνύματα να διαχειριστεί, "κοιμάται" εθελοντικά για ελάχιστο χρονικό διάστημα (π.χ. 0.01 δευτερόλεπτα), αποδεσμεύοντας πλήρως τον πυρήνα και επιτρέποντας στα νήματα των εργατών να προχωρήσουν απρόσκοπτα με την εκτέλεση του κώδικά τους. Παρά αυτή την τεχνική, παραμένει μια εγγενής πραγματικότητα: όσο αυξάνονται τα ενεργά threads που επιχειρούν να εκτελέσουν bytecode, η απόδοση θα επηρεάζεται αναπόφευκτα από το GIL [2].

5.3 Μηχανισμός Εξισορρόπησης Φόρτου

Η καρδιά της απόδοσης του torcpry σε ετερογενή και μη-προβλέψιμα υπολογιστικά περιβάλλοντα εδράζεται στον αλγόριθμο εξισορρόπησης φόρτου και στην πολιτική αναζήτησης και κλοπής εργασιών (**Work Stealing**). Ο κύκλος ζωής ενός αδρανούς νήματος-εργάτη (`idle worker thread`) ορίζεται από μια αυστηρή ακολουθία ενεργειών.

Όταν ένα νήμα-εργάτης (`Worker Thread`) ολοκληρώσει μια εργασία και βρεθεί χωρίς δουλειά, η πρώτη του κίνηση είναι να αναζητήσει νέα εργασία στον δικιά του, τοπική ουρά. Συγκεκριμένα, στοχεύει την υψηλότερη σε ιεραρχία τοπική ουρά η οποία δεν είναι κενή (`highest-level non-empty local queue`). Εάν δεν υπάρχει καμία διαθέσιμη εργασία και όλες οι τοπικές ουρές είναι άδειες, τότε ο εργάτης προσπαθεί να κλέψει κάποια εργασία από κάποια άλλη απομακρυσμένη ουρά εργασιών [2].

Σε αυτό το στάδιο, το αδρανές νήμα-εργάτης στέλνει ένα σύγχρονο αίτημα κλοπής (`synchronous stealing request`) προς το νήμα-εξυπηρετητή (`Server Thread`) της επόμενης στη σειρά διεργασίας MPI, σύμφωνα με το αναγνωριστικό κατάταξής της (`MPI rank`). Εάν η επόμενη διεργασία δεν έχει διαθέσιμη εργασία, ο εργάτης προχωρά στην επόμενη της επόμενης, δημιουργώντας ουσιαστικά μια κυκλική σάρωση (`ring topology`) όλου του MPI cluster. Η διαδικασία αυτή συνεχίζεται ακατάπαυστα μέχρι να εντοπιστεί και να επιστραφεί μια διαθέσιμη εργασία ή μέχρι να ολοκληρωθεί η σάρωση όλων ανεξαιρέτως των διαθέσιμων διεργασιών του συστήματος [2].

Από την άλλη πλευρά της εξίσωσης, όταν το νήμα-εξυπηρετητή μιας διεργασίας λάβει ένα τέτοιο αίτημα κλοπής από ένα αδρανές νήμα-εργάτη άλλου κόμβου, ακολουθεί μια εντελώς αντίστροφη λογική εύρεσης: ξεκινά την αναζήτηση για διαθέσιμη εργασία

από την κατώτερη σε ιεραρχία τοπική ουρά του (lowest-level local queue). Με άλλα λόγια, προσφέρει πρώτα τις εργασίες που βρίσκονται στο πιο εσωτερικό επίπεδο του παραλληλισμού (innermost level of parallelism) [2].

Η προεπιλεγμένη στρατηγική γέννησης εργασιών του *torcpy* (default task spawning policy) προβλέπει τη διανομή των εργασιών του "πρώτου επιπέδου" με κυκλικό τρόπο (cyclically) σε όλους τους διαθέσιμους εργάτες του συστήματος, διασφαλίζοντας την αρχική ευρεία διασπορά του φόρτου. Αντίθετα, τις εργασίες των "εσωτερικών επιπέδων" (inner-level tasks), οι οποίες προκύπτουν από τον ένθετο παραλληλισμό (nested-parallelism), τις υποβάλλει αυστηρά τοπικά στον ίδιο τον κόμβο που τις γέννησε [2].

Ο συνδυασμός αυτής της ασύμμετρης στρατηγικής δημιουργεί ένα εξαιρετικά ευφύες μοτίβο συμπεριφοράς (intelligent structural pattern). Ευνοεί ρητά την κλοπή "χονδροειδών" (coarse-grained) εργασιών μεταξύ των κόμβων (internode stealing), οι οποίες έχουν επαρκές μέγεθος υπολογισμού για να δικαιολογήσουν το χρονικό κόστος της μεταφοράς τους μέσω του δικτύου. Ταυτόχρονα, διατηρεί την εκτέλεση των "λεπτόκοκκων" (fine-grained), βαθύτερων επιπέδων παραλληλισμού στην τοπική μνήμη του εκάστοτε κόμβου, μειώνοντας δραστικά την άσκοπη επικοινωνία (communication overhead) μεταξύ των διεργασιών [2].

5.4 Προγραμματιστική Διεπαφή (API) και Υποστήριξη Κληρονομημένου Κώδικα (SPMD)

Το *torcpy* σχεδιάστηκε για να παρέχει μια Διεπαφή Προγραμματισμού Εφαρμογών (Application Programming Interface - API) που να είναι άμεσα αναγνωρίσιμη σε οποιονδήποτε προγραμματιστή έχει χρησιμοποιήσει τη βιβλιοθήκη `concurrent.futures` της Python, ακολουθώντας αυστηρά τις προδιαγραφές του PEP 3148. Πέραν αυτού, εισάγει εξειδικευμένες ρουτίνες για τη ρύθμιση του συστήματος, τον έλεγχο των καταστάσεων λειτουργίας, τον έλεγχο της κλοπής και την ανάκριση (querying) του περιβάλλοντος χρόνου εκτέλεσης [2].

5.4.1 Βασικές Ρουτίνες Διαχείρισης (Library Routines)

Το κεντρικό API περιλαμβάνει τις ακόλουθες θεμελιώδεις ρουτίνες [2]:

- **start(f)**: Αυτή είναι η λειτουργία αρχικοποίησης (initialization). Πρόκειται για μια συλλογική συνάρτηση (collective function) του MPI, που σημαίνει ότι πρέπει υποχρεωτικά να κληθεί από όλες τις διεργασίες εντός του μπλοκ `__main__` του προγράμματος. Ενεργοποιεί τη βιβλιοθήκη *torcpy* και, στη συνέχεια, εκκινεί τη συνάρτηση `f()` αποκλειστικά στη διεργασία με βαθμίδα μηδέν (rank 0). Η συνάρτηση

αυτή αναλαμβάνει το ρόλο της πρωτεύουσας εργασίας (primary application task). Όταν το δέντρο εκτέλεσης ολοκληρωθεί και η *f()* επιστρέψει, η ρουτίνα *start* φροντίζει για τον ασφαλή και καθαρό τερματισμό (shutdown) ολόκληρης της βιβλιοθήκης σε όλο το cluster.

- **submit(*f*, **args*, *qid*=-1, *callback*=None, *kwargs*):** Είναι ο πρωταρχικός τρόπος με τον οποίο ο χρήστης δημιουργεί νέες εργασίες. Υποβάλλει μία νέα εργασία (task), το οποίο αντιστοιχεί στην ασύγχρονη εκτέλεση της συνάρτησης *f()* με τα παρέχοντα ορίσματα *args*. Η παράμετρος *qid* καθορίζει ρητά το αναγνωριστικό του εργάτη στον οποίο πρέπει να υποβληθεί η εργασία. Εάν το *qid* παραμείνει στην προεπιλεγμένη τιμή του -1, η βιβλιοθήκη προβαίνει αυτόματα σε κυκλική διανομή (cyclic distribution) της εργασίας στο σύνολο των διαθέσιμων εργατών. Μια εξαιρετικά χρήσιμη παράμετρος είναι η *callback*. Όταν παρέχεται, η συνάρτηση *callback* (συνάρτηση επανάκλησης) ενεργοποιείται και εκτελείται αυτόματα στον υπολογιστικό κόμβο (rank) που είχε αρχικά γεννήσει (spawned) την εργασία, τη στιγμή που η εν λόγω εργασία ολοκληρώνεται πλήρως και τα αποτελέσματά της καταφθάνουν πίσω στον κόμβο αφετηρίας.
- **map(*f*, **seq*, *chunksize*=1):** Μια από τις πιο συχνά χρησιμοποιούμενες συναρτήσεις της Python, η *map*, παραλληλοποιείται εγγενώς. Αναλαμβάνει να εκτελέσει τη συνάρτηση *f()* σε όλα τα στοιχεία μιας τεράστιας λίστας/ακολουθίας (sequence) ορισμάτων. Επιστρέφει μια συλλογή με τα αποτελέσματα όλων των παραχθέντων εργασιών. Η παράμετρος *chunksize* (μέγεθος τμήματος) επιτρέπει την ομαδοποίηση πολλών στοιχείων της ακολουθίας σε μία μόνο εργασία, γεγονός το οποίο ελαχιστοποιεί κατακόρυφα το διοικητικό κόστος (overhead) σε περιπτώσεις όπου η εκτέλεση της συνάρτησης διαρκεί κλάσματα του δευτερολέπτου, προστατεύοντας το σύστημα από συμφόρηση.
- **wait(*tasks*=None):** Μέσω αυτής της κλήσης, η τρέχουσα εργασία αναστέλλει τη λειτουργία της και τίθεται σε αναμονή, περιμένοντας να ολοκληρωθούν όλες οι υποεργασίες (child tasks) που ενδεχομένως γέννησε προηγουμένως. Το πιο καινοτόμο στοιχείο της ρουτίνας *wait* στο *torcpy* είναι ότι η εργασία δεν μπλοκάρει άσκοπα τον επεξεργαστή. Το υποκείμενο νήμα εργάτη αποδεσμεύεται προσωρινά από τη συγκεκριμένη εργασία και είναι ελεύθερο να επιστρέψει στην ουρά για να εντοπίσει και να εκτελέσει άλλες διαθέσιμες εργασίες. Η στρατηγική αυτή είναι η απάντηση του *torcpy* στην πρόληψη των αδιεξόδων (deadlocks) που μαστιάζουν τα συμβατικά συστήματα.

5.4.2 Μοντέλο Εκτέλεσης SPMD

Ένα από το σημαντικά χαρακτηριστικό του API είναι η ρουτίνα `spmd(f, *args)`. Ο κλασικός παραλληλισμός εργασιών ακολουθεί το αυστηρό μοντέλο master-worker, δημιουργώντας έναν κλειστό κόσμο όπου οι εργάτες αγνοούν τη δομή του MPI. Το torcpy έρχεται να ανατρέψει αυτή την απομόνωση λειτουργώντας "ορθογώνια" προς το MPI [2].

Η κλήση `spmd` πυροδοτεί άμεσα και ταυτόχρονα την εκτέλεση της συνάρτησης `f()` σε όλες ανεξαιρέτως τις διεργασίες MPI του συστήματος. Αυτό μεταφράζεται σε μια δυναμική μετάβαση (dynamic switching) του περιβάλλοντος. Το σύστημα σταματά να λειτουργεί ως master-worker και μετασχηματίζεται ακαριαία στο κανονικό, θεμελιώδες μοντέλο **SPMD (Single Program Multiple Data)** που υποστηρίζει εγγενώς το MPI [2].

Αυτή η ικανότητα σημαίνει ότι οι χρήστες έχουν πλέον το ελεύθερο να ενσωματώσουν άμεσα παλαιό (legacy) κώδικα MPI, γραμμένο μέσω `mpi4py`, απευθείας μέσα στην εκτέλεση της εργασίας τους. Για παράδειγμα, δύνανται να εκτελέσουν βαριές, συλλογικές λειτουργίες του MPI (MPI collective operations) όπως οι Broadcast, Scatter, και Gather μεταξύ των κόμβων, για να προετοιμάσουν μεγάλα σύνολα δεδομένων πριν καν ξεκινήσει η αναλυτική φάση του συστήματος. Στο τέλος της εκτέλεσης της `spmd`, η εφαρμογή επιστρέφει ομαλά στο task-based μοντέλο [2].

Επιπρόσθετα, το σύστημα προσφέρει μια σειρά βοηθητικών συναρτήσεων (helper functions) για την παρακολούθηση της κατάστασης του υλικού: `torcpy.node_id()` (για να εντοπίσει το rank του κόμβου), `torcpy.worker_id()` (για το συγκεκριμένο νήμα), `torcpy.num_local_workers()`, `torcpy.gettime()` (για χρονομέτρηση ακριβείας), και φυσικά τις ρουτίνες `enable_stealing()` και `disable_stealing()` για το δυναμικό άνοιγμα ή κλείσιμο του μηχανισμού κλοπής [22].

5.5 Παράδειγμα Χρήσης του torcpy

Στην παρακάτω ενότητα παρουσιάζεται ένα απλό παράδειγμα χρήσης του torcpy, όπως και για το προηγούμενο παράδειγμα με το StarPU, με σκοπό να γίνει κατανοητή η βασική λογική λειτουργίας και χρήσης του συστήματος.

Στο παράδειγμα αυτό, δημιουργούνται 8 εργασίες που εκτελούν μια συνάρτηση `busy_work()` η οποία περιέχει έναν βρόχο που καταναλώνει CPU για 8 δευτερόλεπτα. Οι εργασίες αυτές υποβάλλονται στο σύστημα torcpy και εκτελούνται παράλληλα σε διαφορετικές διεργασίες και νήματα-εργάτες. Η εκτέλεση των εργασιών μπορεί να παρακολουθηθεί σε πραγματικό χρόνο με την χρήση του εργαλείου `htop` σε linux σύστημα, το οποίο εμφανίζει τα νήματα/διεργασίες και τη χρήση της CPU.. Μετά την ολοκλήρωση των εργασιών, εμφανίζεται ο συνολικός χρόνος εκτέλεσης.

Θεωρούμε ότι το σύστημα torcpy έχει εγκατασταθεί και είναι διαθέσιμο στο πε-

ριβάλλον Python. Οδηγίες για την εγκατάσταση του torcpy μπορούν να βρεθούν εδώ [22].

Αρχικά πρέπει να εισάγουμε το module `torcpy` μαζί με τις απαραίτητες βιβλιοθήκες και να ορίσουμε μια συνάρτηση που θα εκτελεί την CPU-bound εργασία. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```
import time
import os
import torcpy as torc

def busy_work(seconds):
    # CPU-bound busy loop to generate measurable load
    t0 = time.time()
    x = 1
    while time.time() - t0 < seconds:
        # cheap integer ops to keep CPU busy
        x = ((x << 1) ^ (x >> 3)) & 0xFFFFFFFF
    return x
```

Για την δημιουργία και υποβολή των εργασιών (tasks), που το καθένα θα εκτελεί την παραπάνω συνάρτηση `busy_work()`, χρησιμοποιούμε μια συνάρτηση, ας την πούμε `main()`. Μέσα σε αυτήν, δημιουργούμε μια λίστα `tasks` για να αποθηκεύσουμε τα αντικείμενα των εργασιών που υποβάλλουμε. Στη συνέχεια, χρησιμοποιούμε έναν βρόχο `for` για να υποβάλλουμε 8 εργασίες, καθεμία από τις οποίες καλεί τη συνάρτηση `busy_work()` με όρισμα 8.0 δευτερόλεπτα. Μετά την υποβολή των εργασιών, καλούμε τη συνάρτηση `torc.wait()` για να περιμένουμε την ολοκλήρωση όλων των εργασιών. Τέλος, συλλέγουμε τα αποτελέσματα των εργασιών και εμφανίζουμε τον συνολικό χρόνο εκτέλεσης. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```
def main():
    print(f"PID: {os.getpid()}, ntasks=8, sec=8.0")
    print("Start submitting tasks...")

    t0 = time.time()
    tasks = []
    for i in range(8):
        tasks.append(torc.submit(busy_work, 8.0))

    # Wait for completion
```

```

torc.wait()
t1 = time.time()

# Collect results (force materialization)
for i, t in enumerate(tasks):
    try:
        _ = t.result()
    except Exception as e:
        print(f"task {i} error: {e}")

elapsed = t1 - t0
print("=" * 60)
print("Framework: torcpy (Monitor dummy workload)")
print(f"Submitted tasks: 8")
print(f"Work per task (s): 8.0")
print(f"Elapsed time: {elapsed:.4f} seconds")
print("=" * 60)

```

Τέλος, για την εκτέλεση της `main()` συνάρτησης χρησιμοποιούμε την εντολή `torc.start(main)`, η οποία εκκινεί το περιβάλλον του torcpy και περιμένει την ολοκλήρωση της `main()` συνάρτησης. Παρακάτω φαίνεται το απόσπασμα κώδικα που περιγράφει την διαδικασία αυτή:

```

if __name__ == '__main__':
    torc.start(main)

```

5.5.1 Ολοκληρωμένος Κώδικας Παραδείγματος

Παρακάτω παρουσιάζεται ο πλήρης κώδικας του παραδείγματος που περιγράφηκε παραπάνω, ο οποίος περιλαμβάνει όλα τα προηγούμενα αποσπάσματα κώδικα:

```

"""
Monitor example for torcpy
- Submits multiple CPU-bound dummy tasks (busy loop) to torcpy
- Use `htop` while it runs to observe processes/threads and CPU usage
"""

import time
import os
import torcpy as torc

```

```
def busy_work(seconds):
    # CPU-bound busy loop to generate measurable load
    t0 = time.time()
    x = 1
    while time.time() - t0 < seconds:
        # cheap integer ops to keep CPU busy
        x = ((x << 1) ^ (x >> 3)) & 0xFFFFFFFF
    return x

def main():
    print(f"PID: {os.getpid()}, ntasks=8, sec=8.0")
    print("Start submitting tasks...")

    t0 = time.time()
    tasks = []
    for i in range(8):
        tasks.append(torc.submit(busy_work, 8.0))

    # Wait for completion
    torc.wait()
    t1 = time.time()

    # Collect results (force materialization)
    for i, t in enumerate(tasks):
        try:
            _ = t.result()
        except Exception as e:
            print(f"task {i} error: {e}")

    elapsed = t1 - t0
    print("=" * 60)
    print("Framework: torcpy (Monitor dummy workload)")
    print(f"Submitted tasks: 8")
    print(f"Work per task (s): 8.0")
    print(f"Elapsed time: {elapsed:.4f} seconds")
    print("=" * 60)

if __name__ == '__main__':
    torc.start(main)
```

Βιβλιογραφία

- [1] Παπαδάκης Στυλιανός και Διαμαντάρας Κωνσταντίνος. *Προγραμματισμός και Αρχιτεκτονική Συστημάτων Παράλληλης Επεξεργασίας*. 1η έκδοση. Αθήνα: Εκδόσεις Κλειδάριθμος, 2012. ISBN: 978-960-461-446-2.
- [2] Panagiotis E Hadjidoukas κ.ά. ``torepy: Supporting task parallelism in Python". Στο: *SoftwareX* 12 (2020), σ. 100517.
- [3] Cédric Augonnet κ.ά. ``StarPU: a unified platform for task scheduling on heterogeneous multicore architectures". Στο: *European Conference on Parallel Processing*. Springer. 2009, σσ. 863–874.
- [4] Wikipedia. *NUMA (Non-Uniform Memory Access)*. 2024. URL: https://en.wikipedia.org/wiki/Non-uniform_memory_access.
- [5] Frank Denneman. *NUMA Deep Dive Part 1: From UMA to NUMA*. 2016. URL: <https://frankdenneman.ai/2016-07-07-numa-deep-dive-part-1-uma-numa/>.
- [6] Super Micro Computer, Inc. *What Is Non-Uniform Memory Access (NUMA)?* 2025. URL: <https://www.supermicro.com/en/glossary/numa>.
- [7] Justin Deters κ.ά. ``A NUMA-Aware Provably-Efficient Task-Parallel Platform Based on the Work-First Principle". Στο: *2018 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2018, σσ. 59–70. DOI: [10.1109/IISWC.2018.8573486](https://doi.org/10.1109/IISWC.2018.8573486). URL: <http://dx.doi.org/10.1109/IISWC.2018.8573486>.
- [8] RapidFork Technology. *Breaking Down Python Concurrency: The Global Interpreter Lock (GIL) and Its Effect on Multi-threading*. 2023. URL: <https://rapidfork.medium.com/breaking-down-python-concurrency-the-gil-and-its-effect-on-multi-threading-b5ab99083cb2>.
- [9] Sam Gross. *PEP 703 -- Making the Global Interpreter Lock Optional in CPython*. Python Enhancement Proposal 703. Python Software Foundation, 2023. URL: <https://peps.python.org/pep-0703/>.

- [10] William Ruys κ.ά. "A deep dive into task-based parallelism in Python". Στο: *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE. 2024, σσ. 1147–1149.
- [11] A.P. Adewole κ.ά. "Understanding and Implementing Concurrency in Python-based Systems". Στο: *International Journal of Engineering Research & Technology (IJERT)* 15.03 (2026). ISSN: 2278-0181. URL: <https://www.ijert.org/understanding-and-implementing-concurrency-in-python-based-systems-ijertv15is031148>.
- [12] StarPU Development Team. *StarPU Handbook for StarPU 1.4.12*. INRIA. 2025. URL: <https://files.inria.fr/starpu/doc/starpu.pdf>.
- [13] Cédric Augonnet, Samuel Thibault και Raymond Namyst. *StarPU: A Runtime System for Scheduling Tasks, or How to Get Portable Performance on Accelerator-Based Platforms Without the Agonizing Pain*. Presentation at NVIDIA GPU Technology Conference (GTC 2010), San Jose, USA. 2010. URL: https://www.nvidia.com/content/GTC-2010/pdfs/2160_GTC2010.pdf.
- [14] Cédric Augonnet, Samuel Thibault και Raymond Namyst. *StarPU: a Runtime System for Scheduling Tasks over Accelerator-Based Multicore Machines*. Research Report RR-7240. INRIA, Μαρ. 2010, σ. 33. URL: <https://inria.hal.science/inria-00467677>.
- [15] Cédric Augonnet κ.ά. "Data-aware task scheduling on multi-accelerator based platforms". Στο: *2010 IEEE 16th international conference on parallel and distributed systems*. IEEE. 2010, σσ. 291–298.
- [16] StarPU Development Team. *StarPU Developer Documentation*. INRIA. 2026. URL: https://files.inria.fr/starpu/doc/starpu_dev.pdf.
- [17] StarPU Development Team. *StarPU: A Unified Runtime System for Heterogeneous Multicore Architectures*. 2026. URL: <https://starpu.gitlabpages.inria.fr/>.
- [18] Alex Aiken. *StarPU -- CS315B Lecture 13*. Lecture slides, Stanford University, CS315B: Parallel Computing. URL: <https://web.stanford.edu/class/cs315b/lectures/lecture13.pdf>.
- [19] Xiaoke Qin και Prabhat Mishra. "Automated Generation of Directed Tests for Transition Coverage in Cache Coherence Protocols". Στο: *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*. 2012. URL: <https://www.cise.ufl.edu/research/cad/Publications/date12.pdf>.

-
- [20] Redis. *Understanding Cache Coherence in Systems*. URL: <https://redis.io/glossary/cache-coherence/>.
- [21] Cédric Augonnet κ.ά. *Getting Started with StarPU*. Tutorial at Symposium on Application Accelerators in High-Performance Computing (SAAHPC 2010), Knoxville, USA. 2010. URL: <https://starpu.gitlabpages.inria.fr/tutorials/2010-07-SAAHPC/saahpc.pdf>.
- [22] IBM. *torc_py: A Platform-Agnostic Adaptive Load Balancing Library for Task Parallelism*. GitHub Repository. URL: https://github.com/ibm/torc_py.

